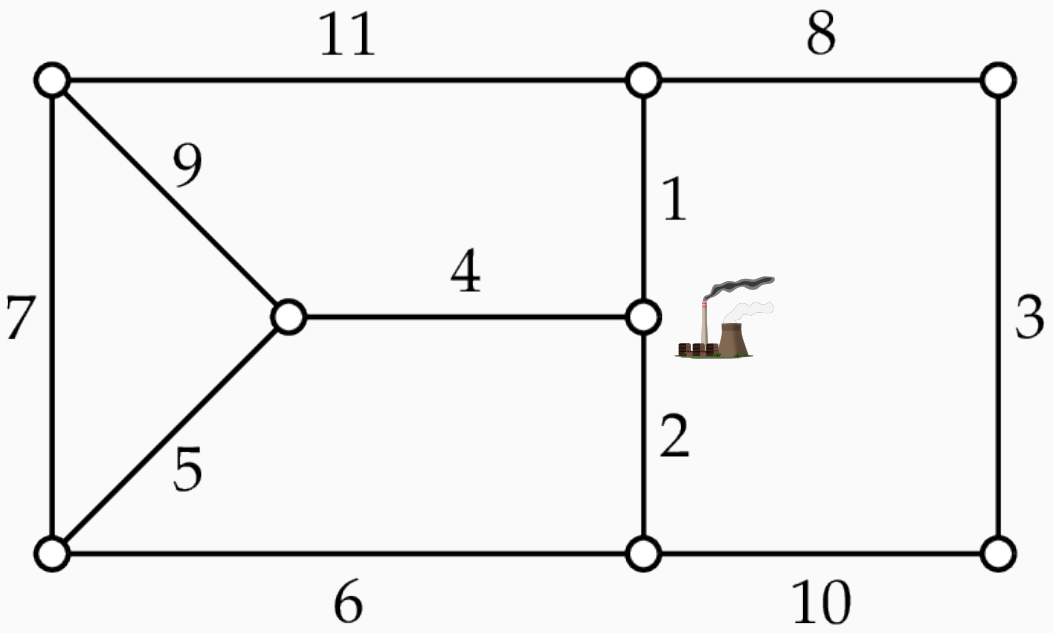
Minimum Spanning Trees

Prof. Timothy Sun

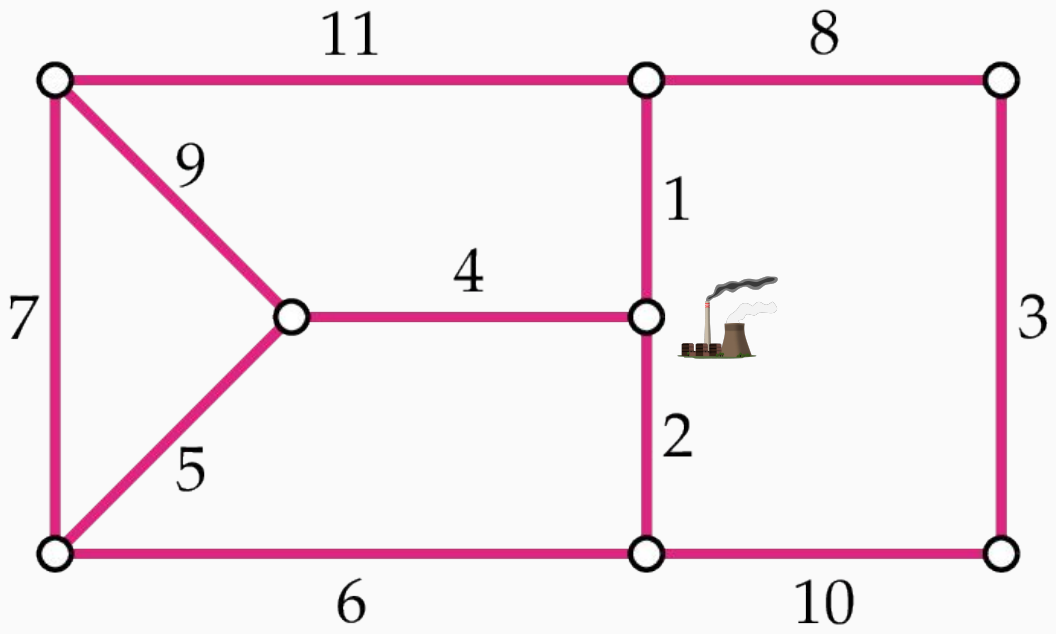
Setting up a power grid

Goal: add power lines to give power to each city



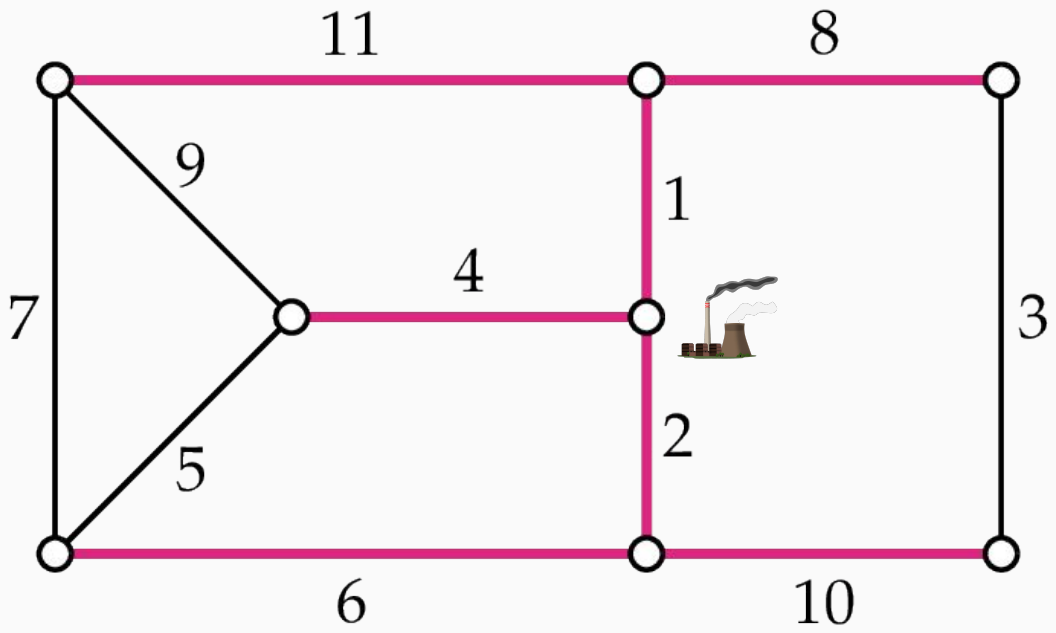
Setting up a power grid

Goal: add power lines to give power to each city



Setting up a power grid

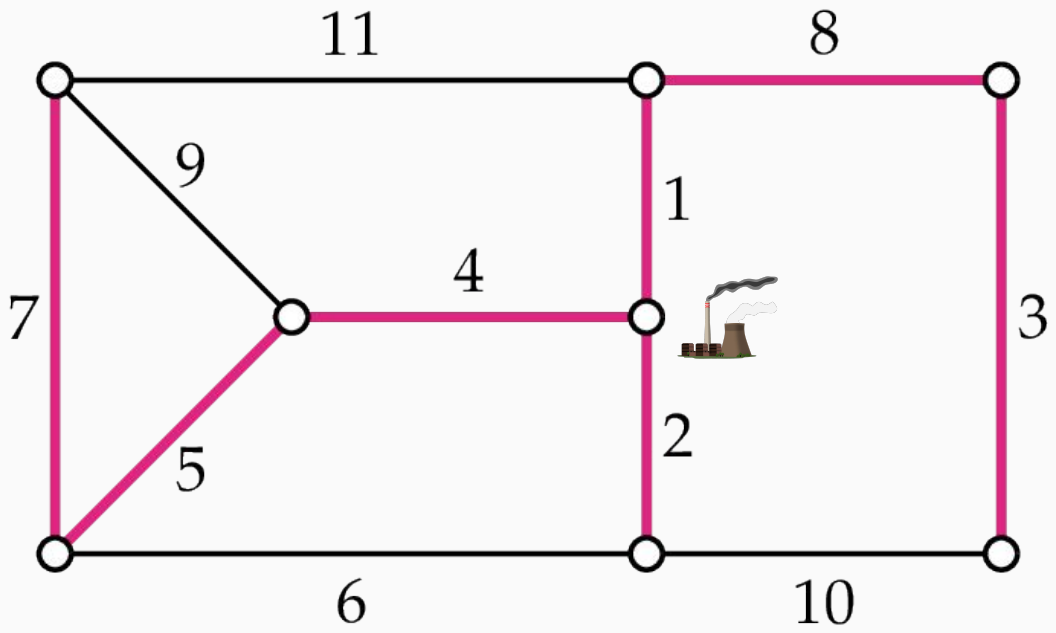
Goal: add power lines to give power to each city, ***minimizing cost***



Only need to include some of the edges to ***maintain connectivity***.

Setting up a power grid

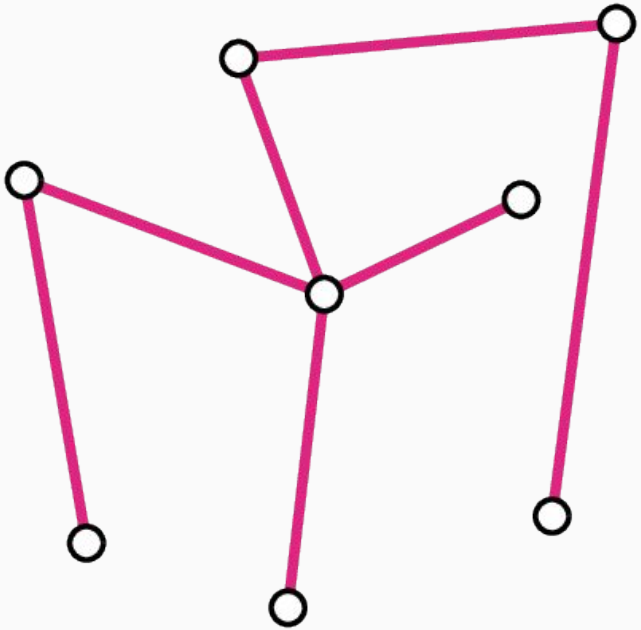
Goal: add power lines to give power to each city, ***minimizing cost***



Only need to include some of the edges to ***maintain connectivity***.

Trees

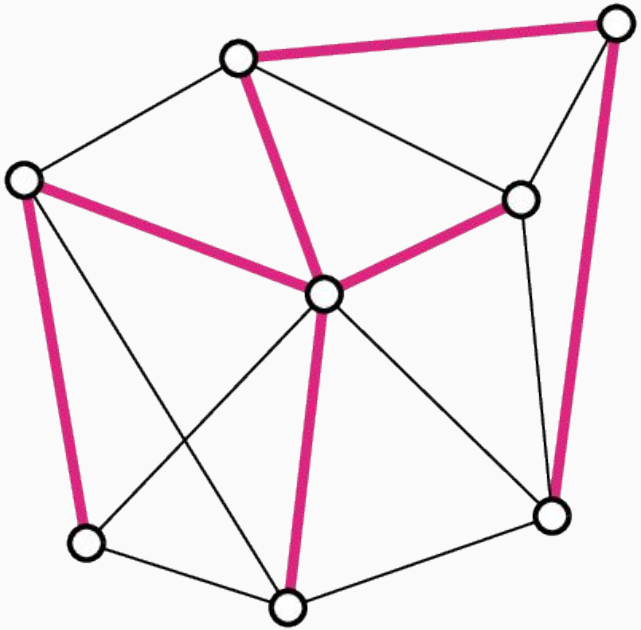
A **tree** is an undirected, ***acyclic***, ***connected*** graph.



Trees

A **tree** is an undirected, ***acyclic***, ***connected*** graph.

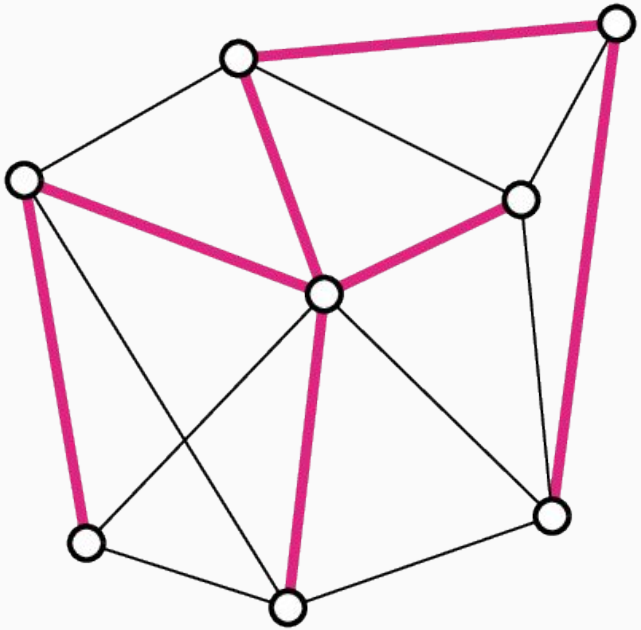
A **spanning tree** is a subgraph on the same vertex set that's a tree.



Trees

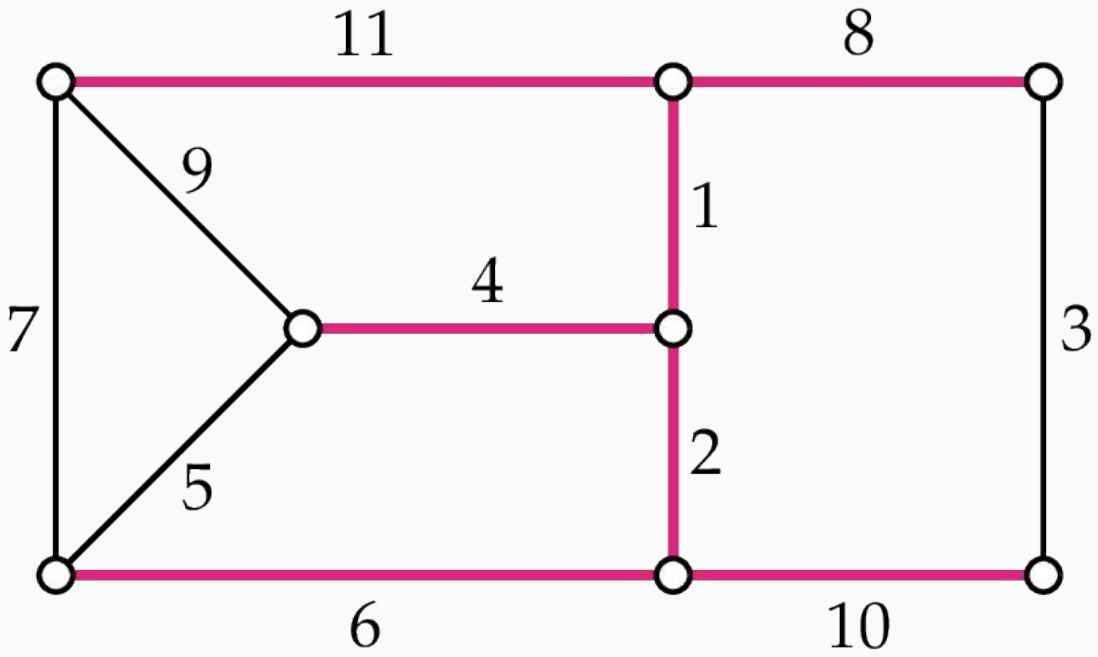
Properties:

- “Minimal connected graph”
- “Maximal acyclic graph”
- $|\boldsymbol{E}| = |\boldsymbol{V}|-1$
- Unique path between pairs of vertices
- etc.



Minimum Spanning Tree

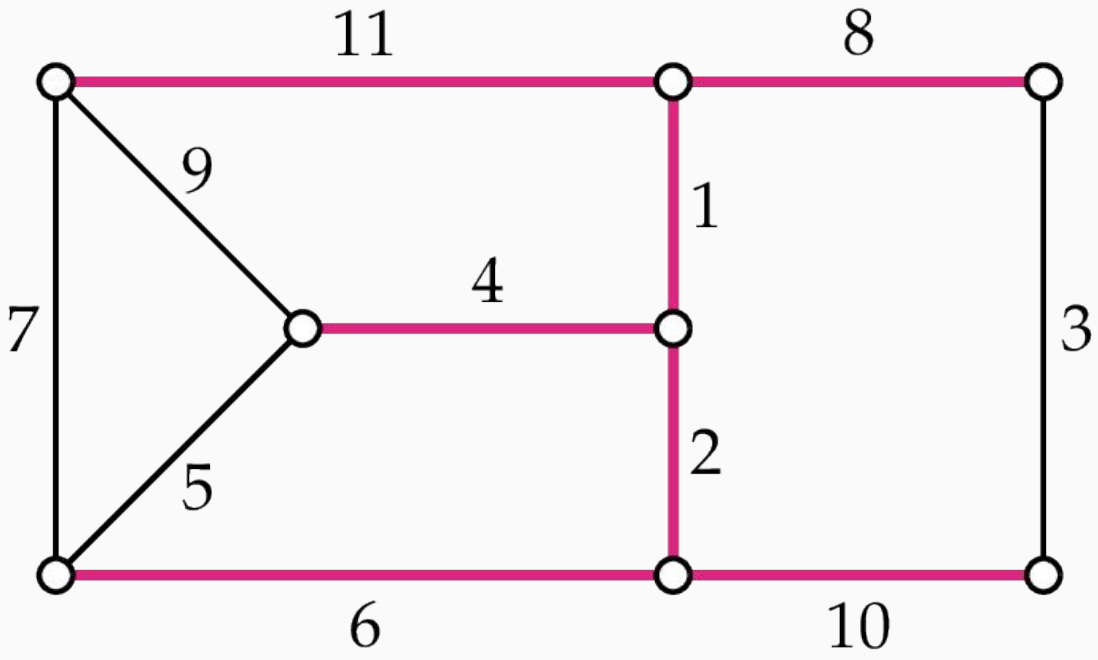
Each edge has a **weight** $w(\mathbf{e})$



Minimum Spanning Tree

Each edge has a **weight** $w(\boldsymbol{e})$

The weight of a tree $w(\boldsymbol{T})$ is the sum of its edge weights



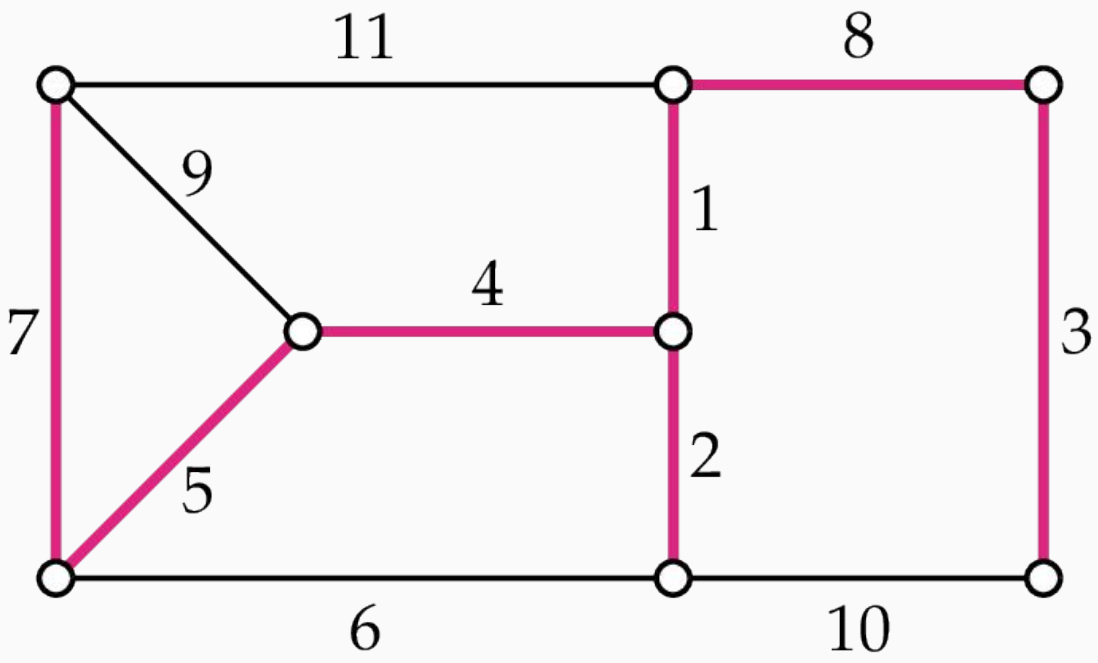
$w(\boldsymbol{T}) = 42$

Minimum Spanning Tree

Each edge has a **weight** $w(\boldsymbol{e})$

The weight of a tree $w(\boldsymbol{T})$ is the sum of its edge weights

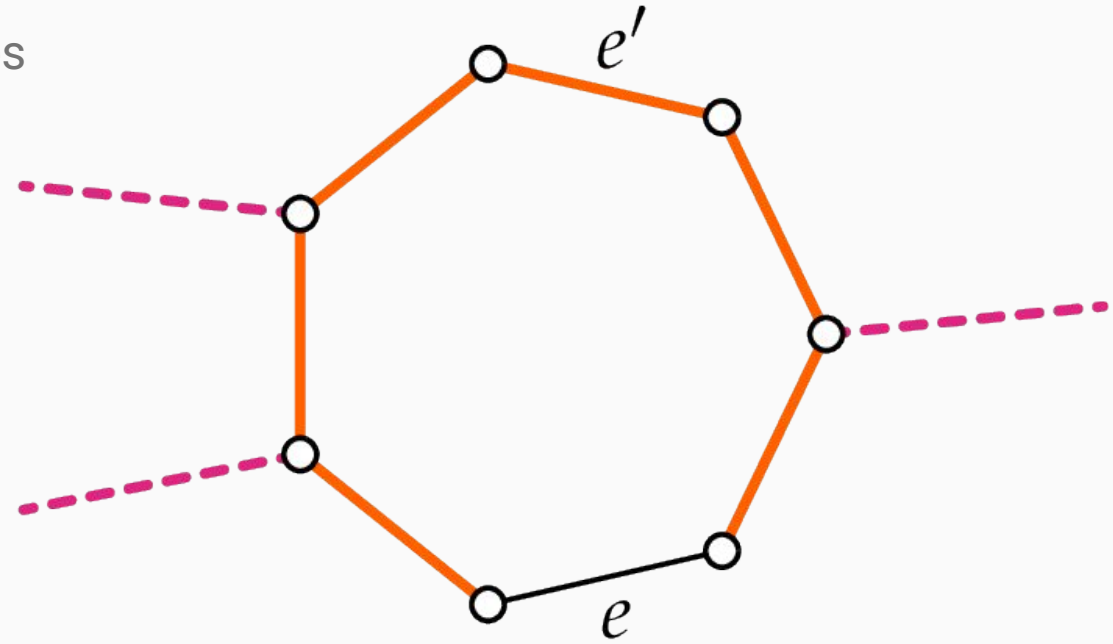
A ***minimum spanning tree*** is a spanning tree with smallest weight



$w(\boldsymbol{T}) = 30$

Fundamental cycles

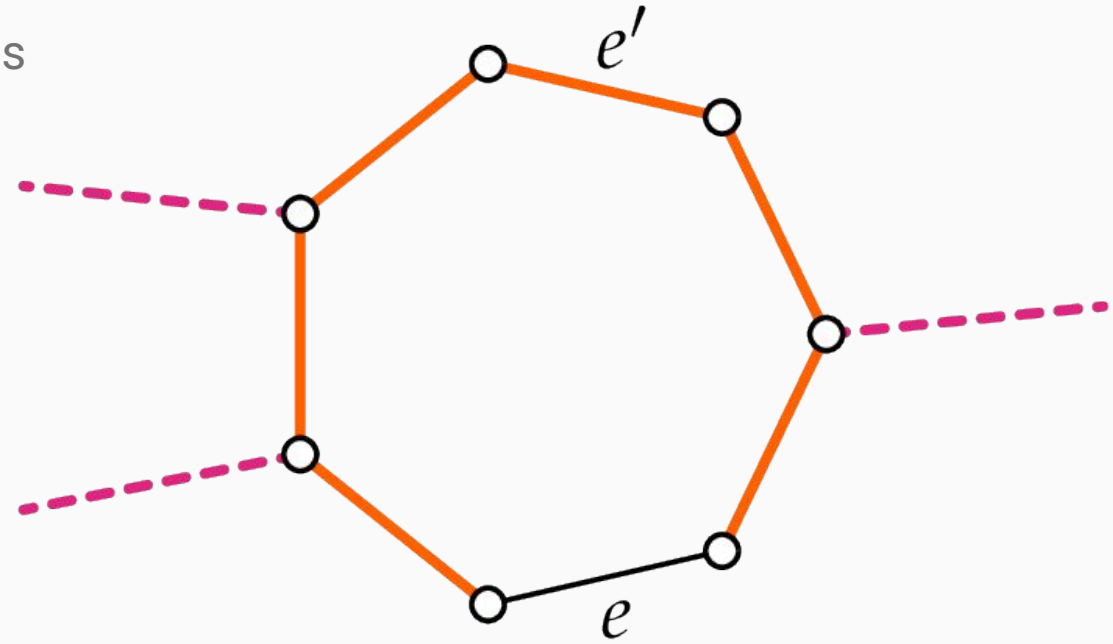
Adding any edge to a tree makes a **fundamental cycle**.



Fundamental cycles

Adding any edge to a tree makes a **fundamental cycle**.

What can we say about $w(\mathbf{e})$ if the tree is an MST?

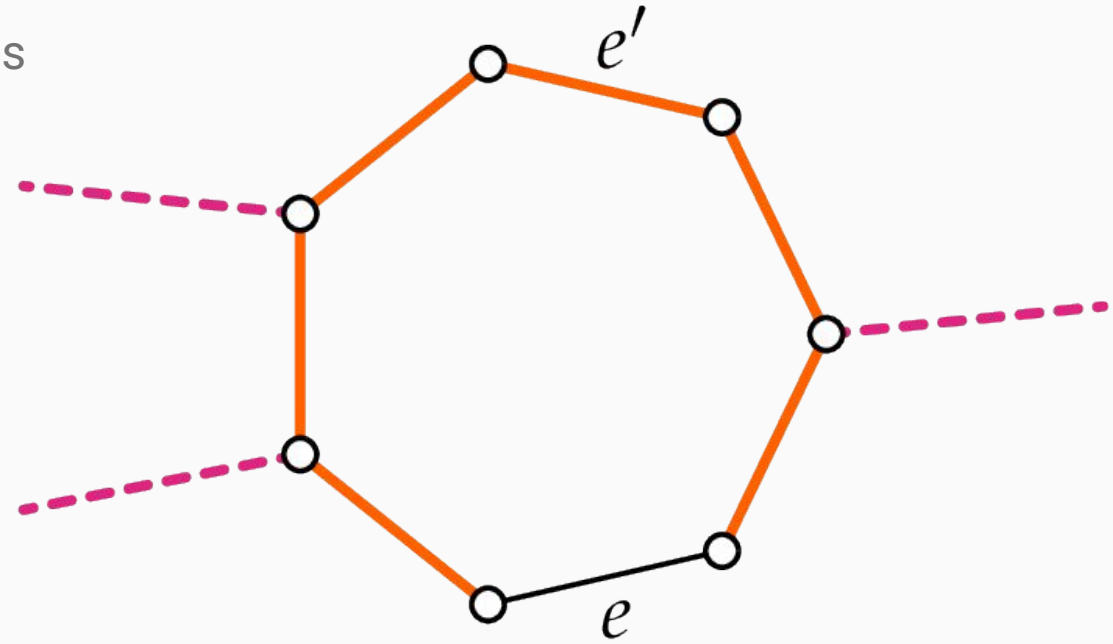


Fundamental cycles

Adding any edge to a tree makes a **fundamental cycle**.

What can we say about $w(\mathbf{e})$ if the tree is an MST?

For every e' on fundamental cycle, $w(e') \leq w(e)$

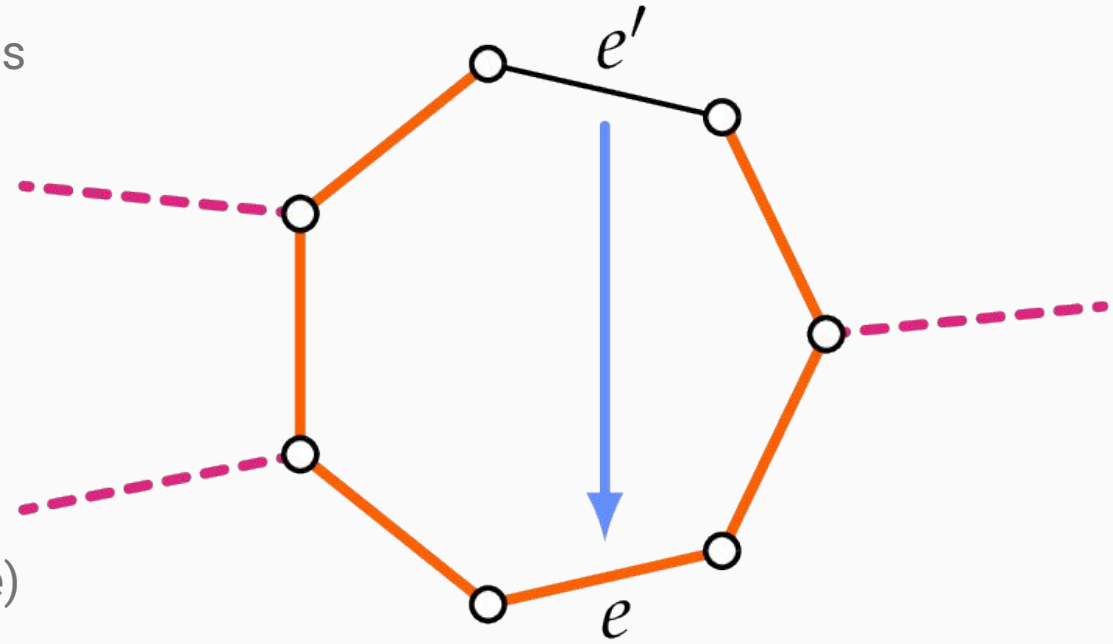


Fundamental cycles

Adding any edge to a tree makes a **fundamental cycle**.

What can we say about w(**e**) if the tree is an MST?

For every e' on fundamental cycle, *w*(e') ≤ *w*(e)
(otherwise, you could **exchange**)



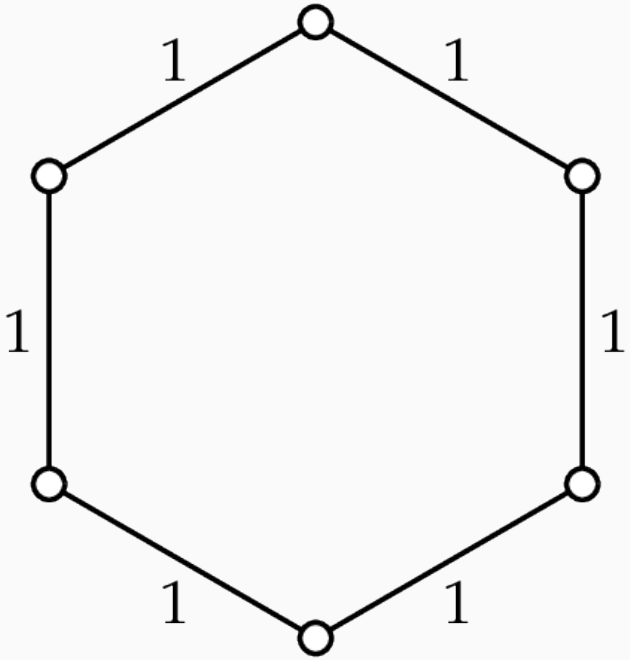
MST Uniqueness

Useful to think of ***the*** MST of a graph…

MST Uniqueness

Useful to think of ***the*** MST of a graph…

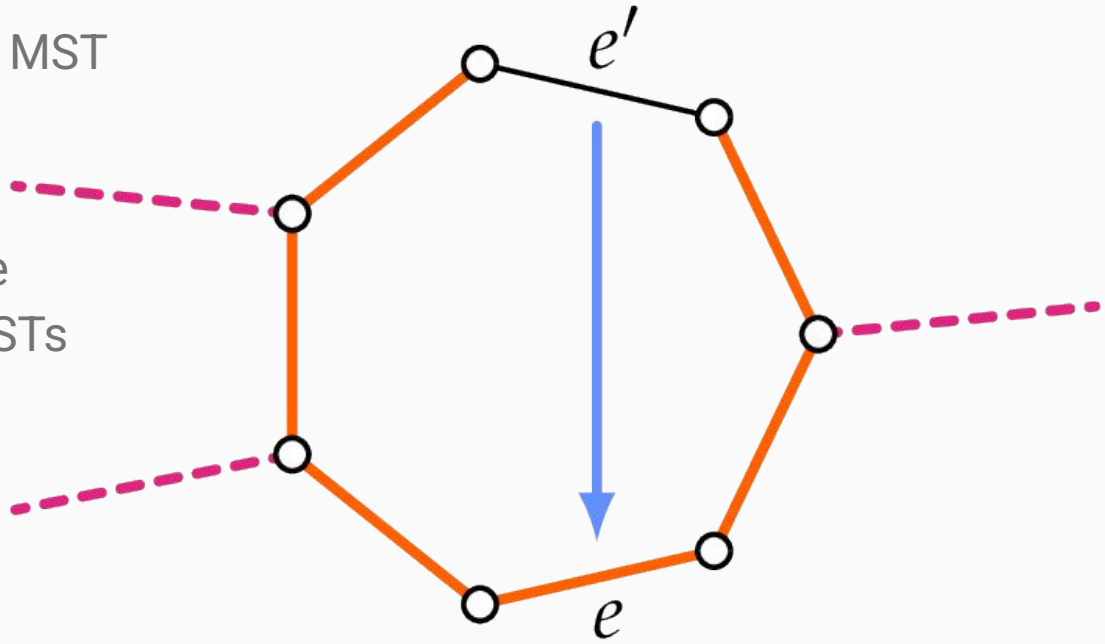
...but there can be more than one.



MST Uniqueness

Distinct edge weights → unique MST

Proof sketch: fundamental cycle flipping between two distinct MSTs



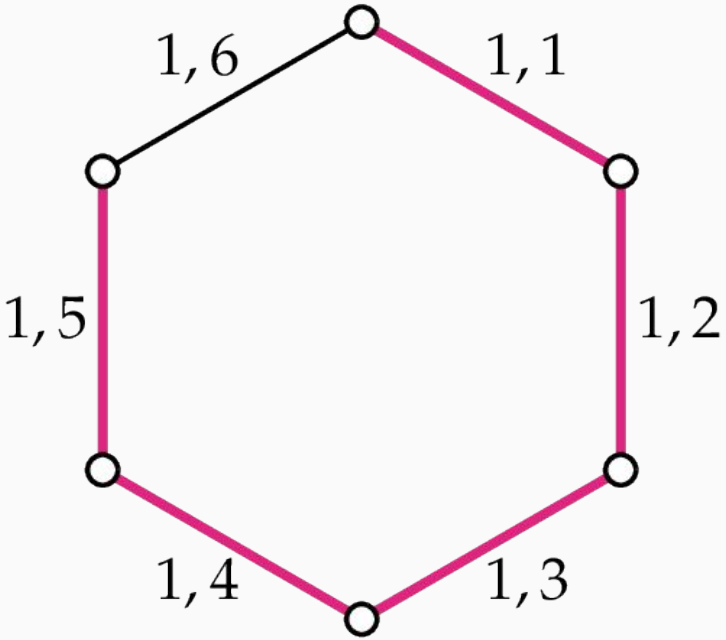
MST Uniqueness

What about duplicate edge weights?

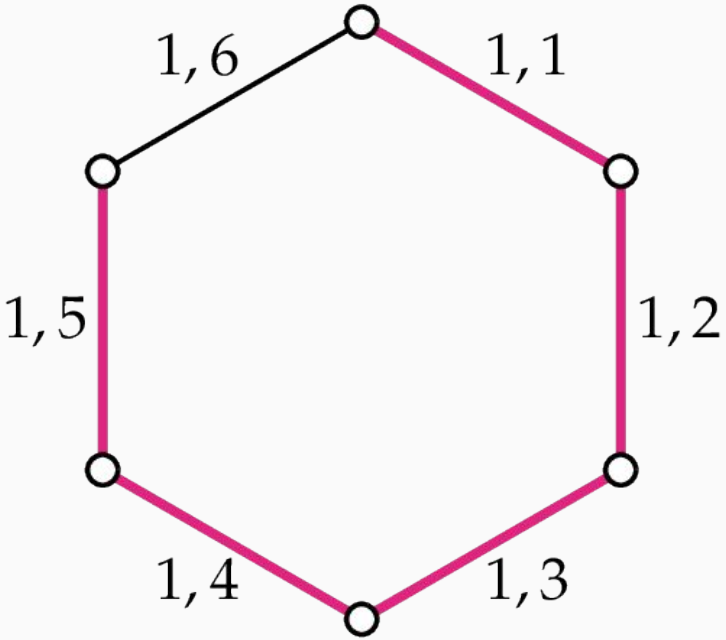
Find a consistent tie-breaking rule, e.g.:

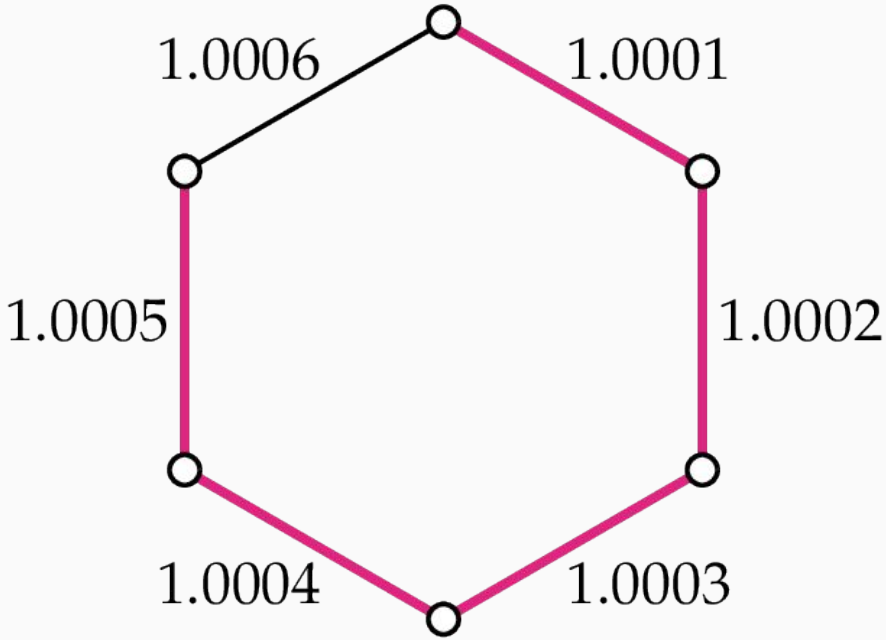
- Order the edges
- Perturb the edge weights

MST Uniqueness



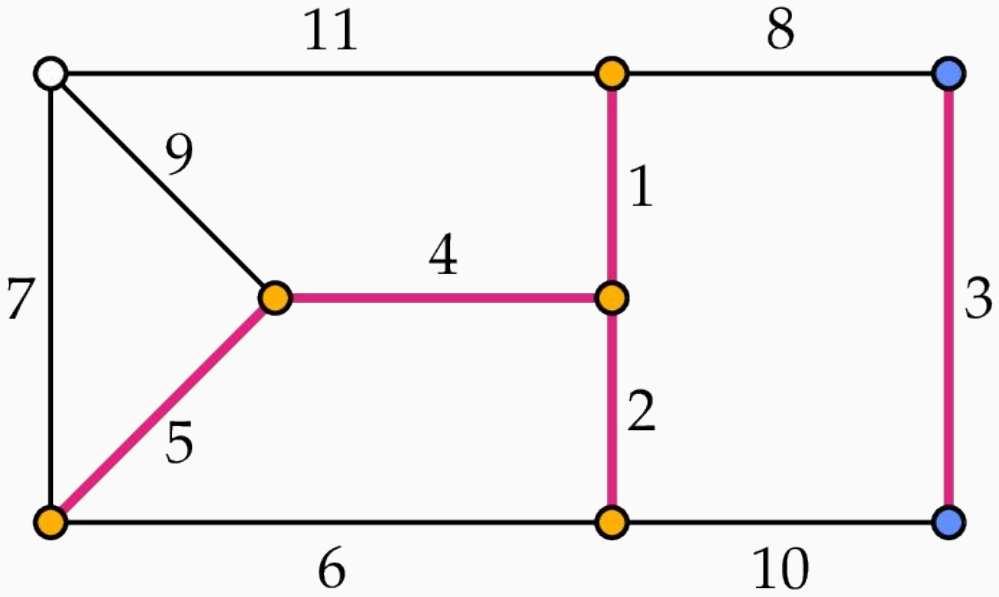
MST Uniqueness





Safe Edges

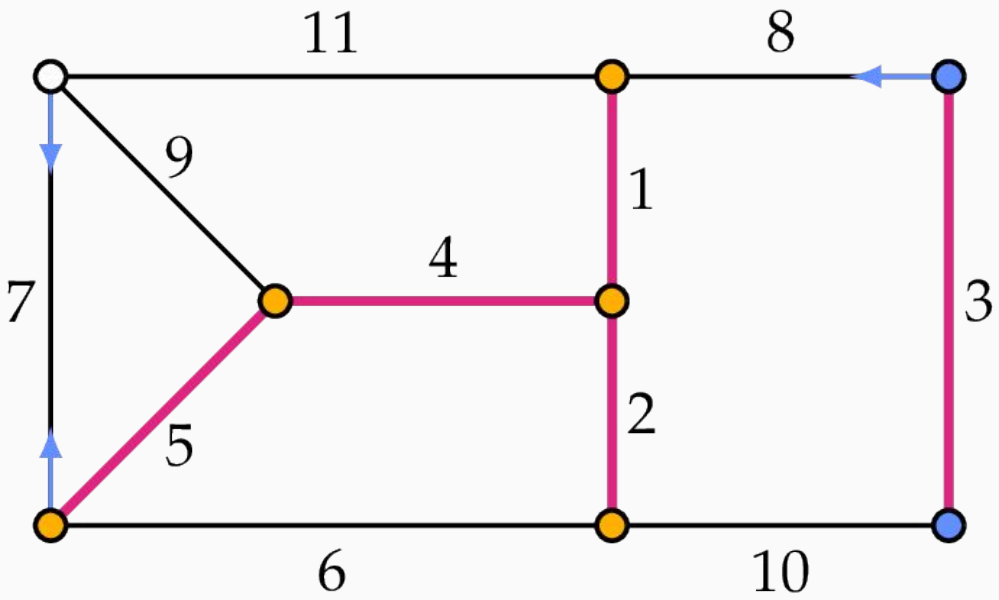
At every iteration, an MST algorithm maintains a “spanning forest” ***F***



Safe Edges

At every iteration, an MST algorithm maintains a “spanning forest” ***F***

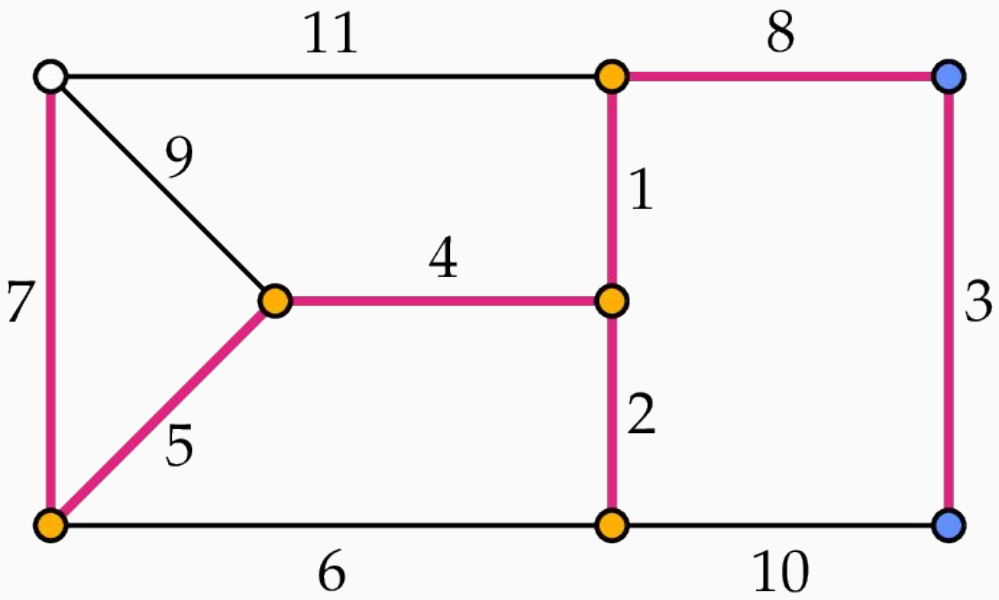
For a component of ***F***, its **safe edge** is the ***lightest*** edge going to a different component



Safe Edges

General idea: “find a safe edge and add to ***F***” (greedy)

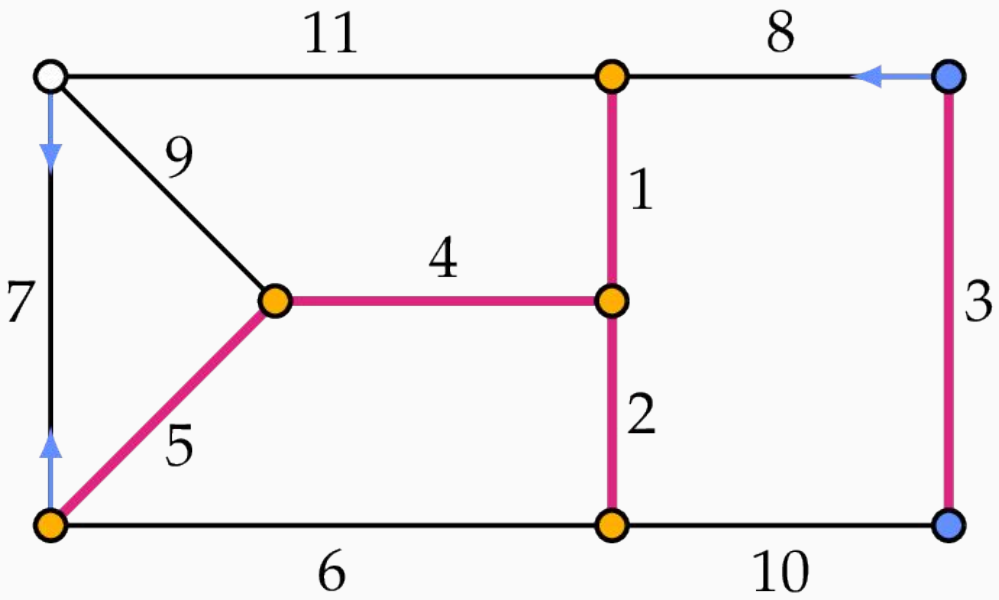
Claim: MST contains all safe edges



Safe Edges

Executive summary:

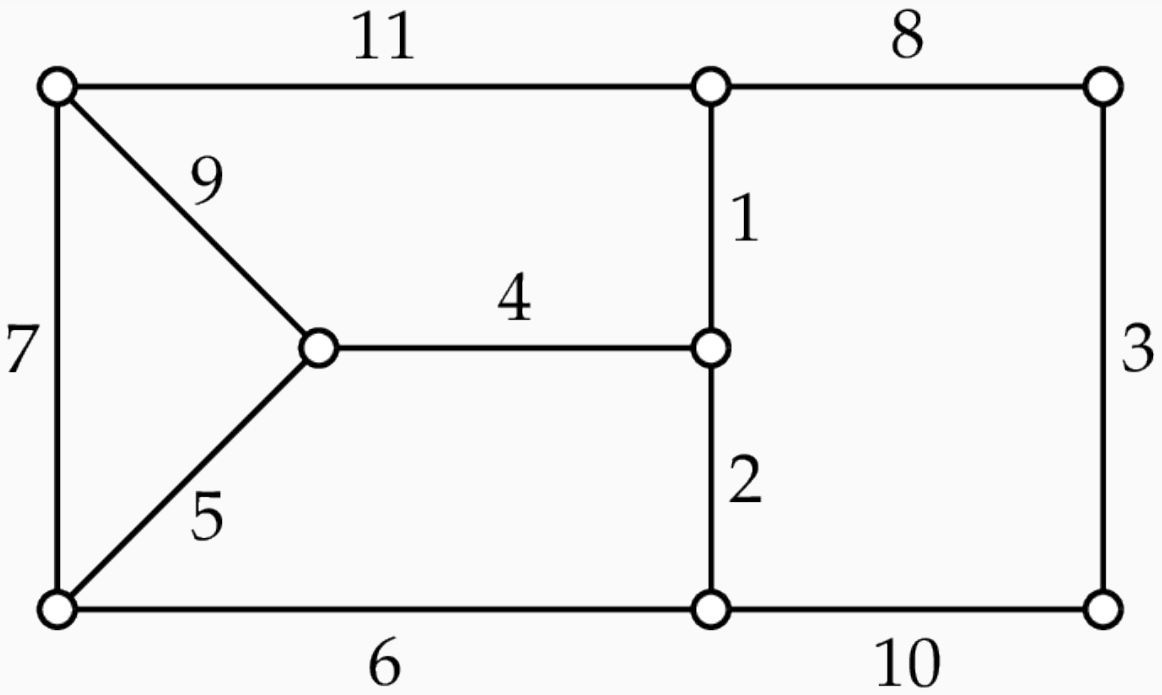
- Kruskal: add lightest safe edge
- Prim: add safe edge of component of ***v***
- Boruvka: add all safe edges at once



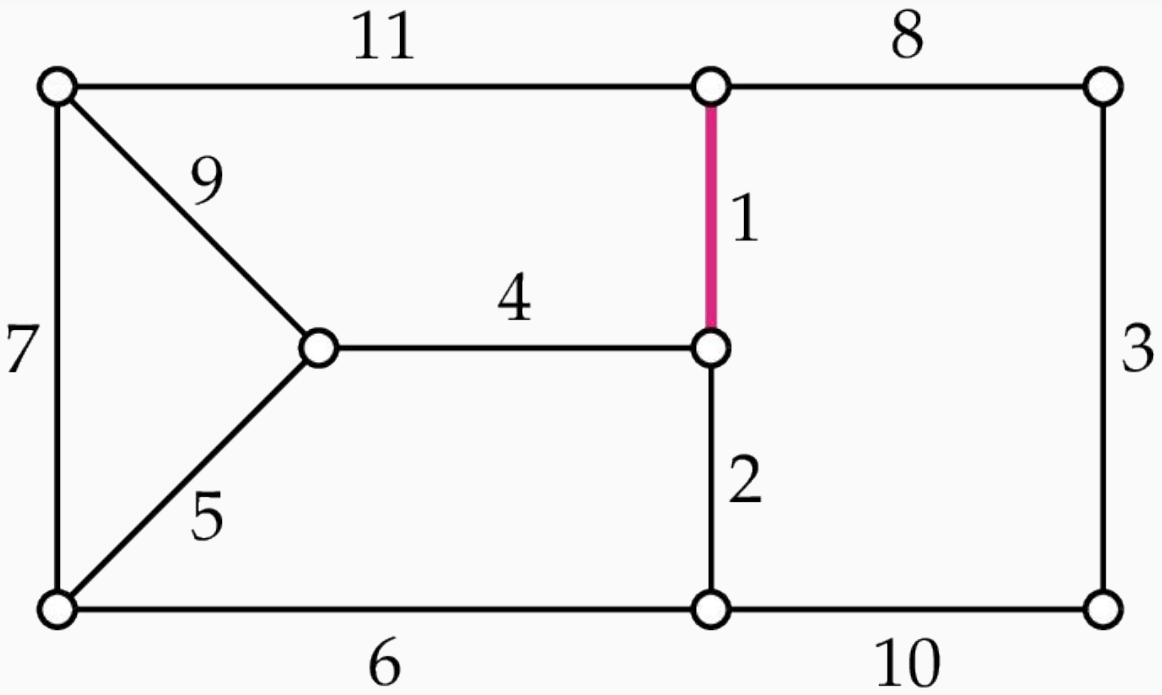
Kruskal’s algorithm

1. Sort all the edge weights.
2. Repeatedly pick the smallest edge without creating a cycle.

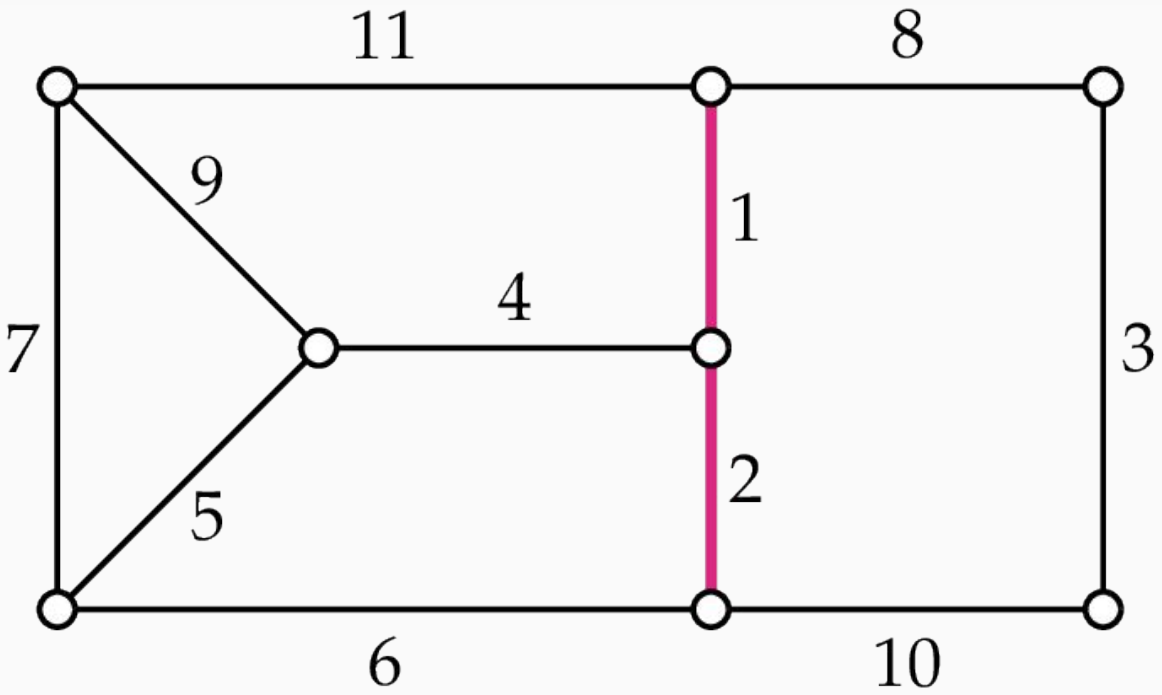
Kruskal’s algorithm



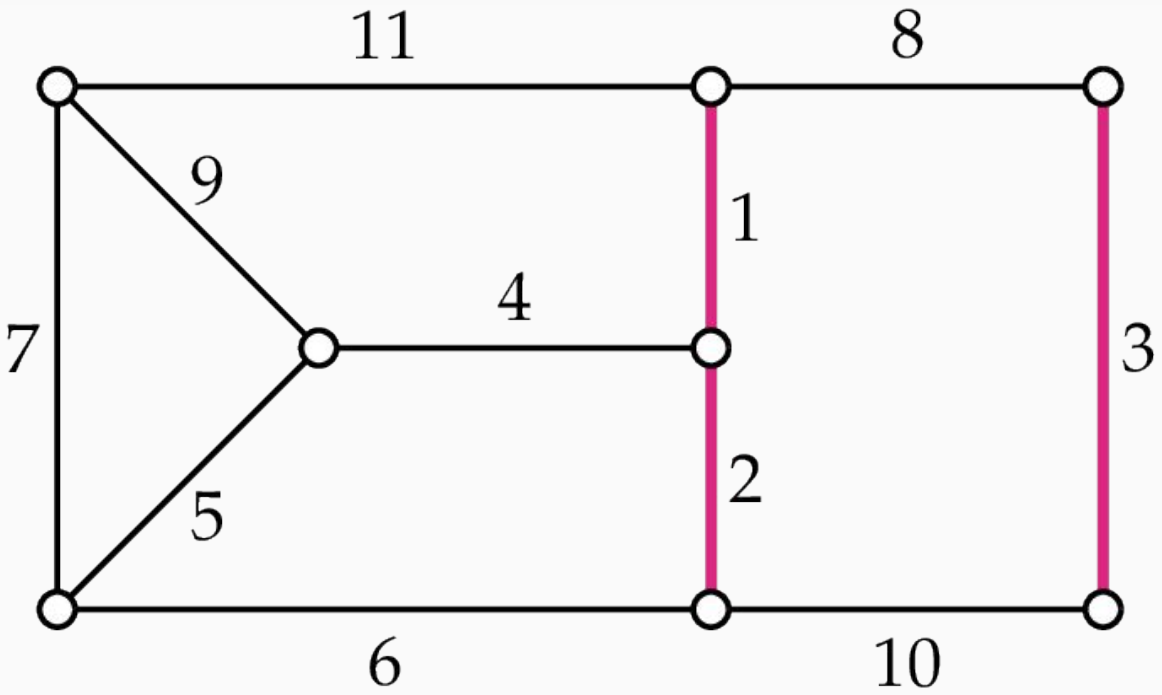
Kruskal’s algorithm



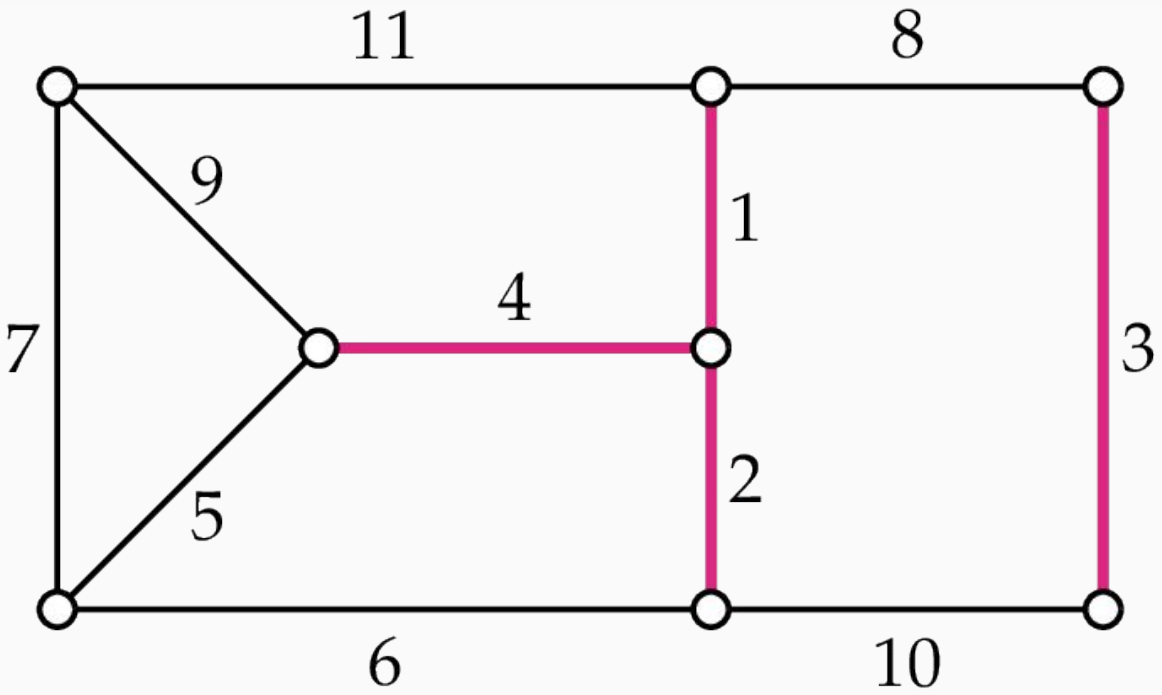
Kruskal’s algorithm



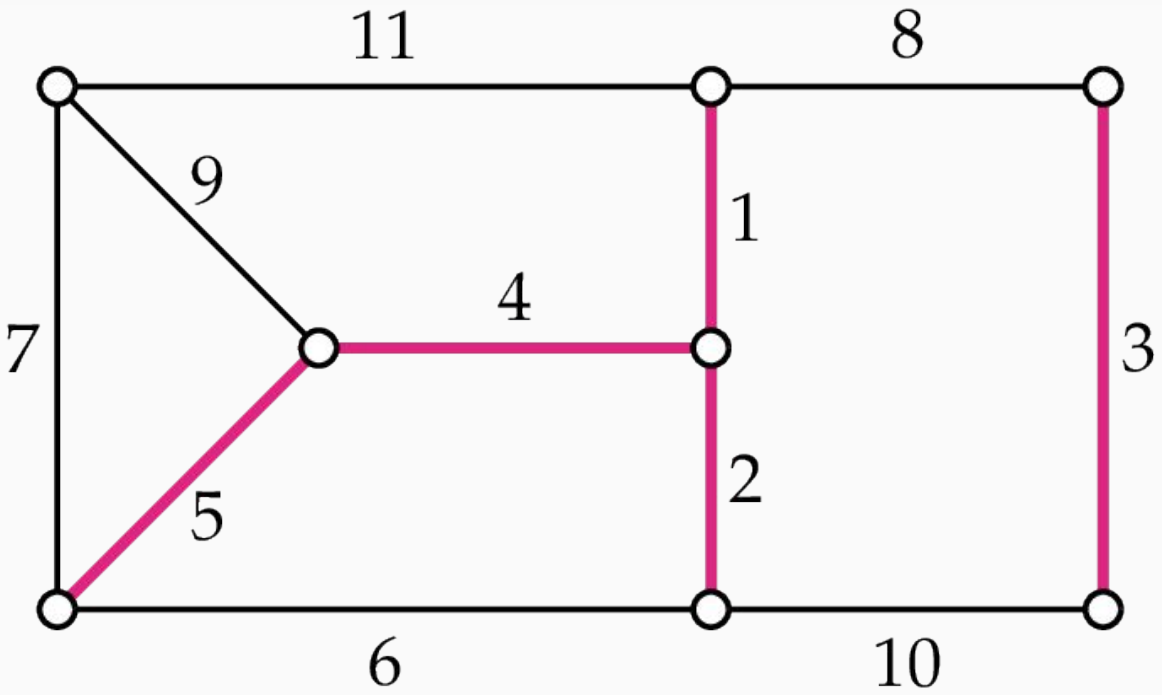
Kruskal’s algorithm



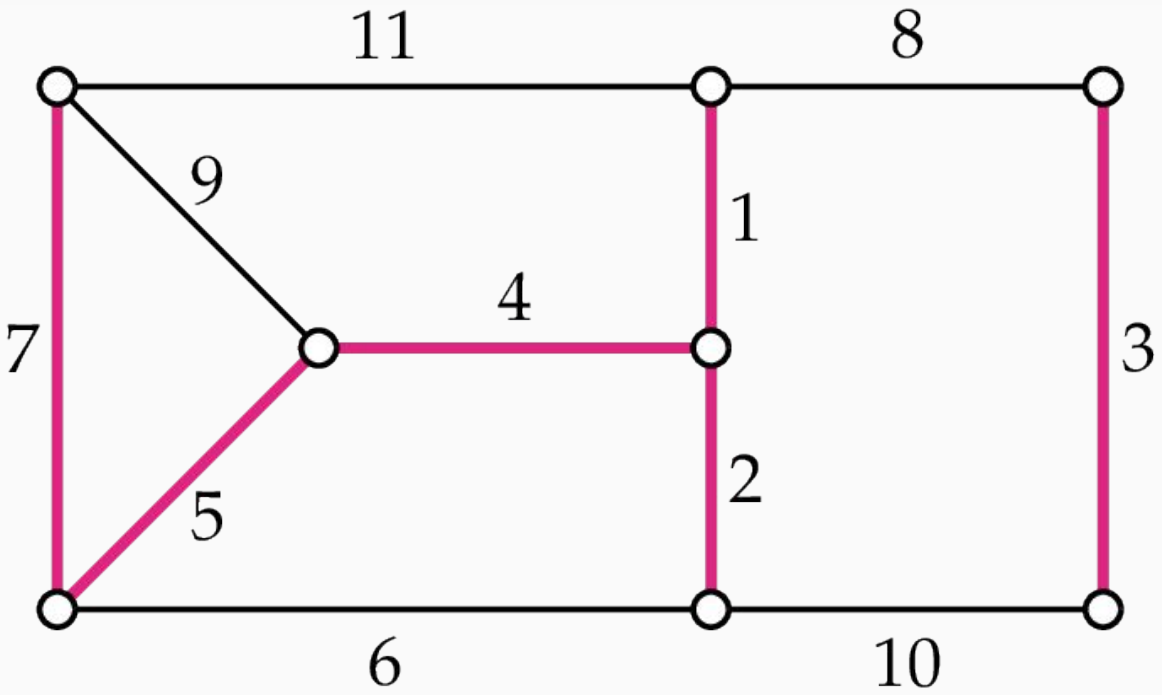
Kruskal’s algorithm



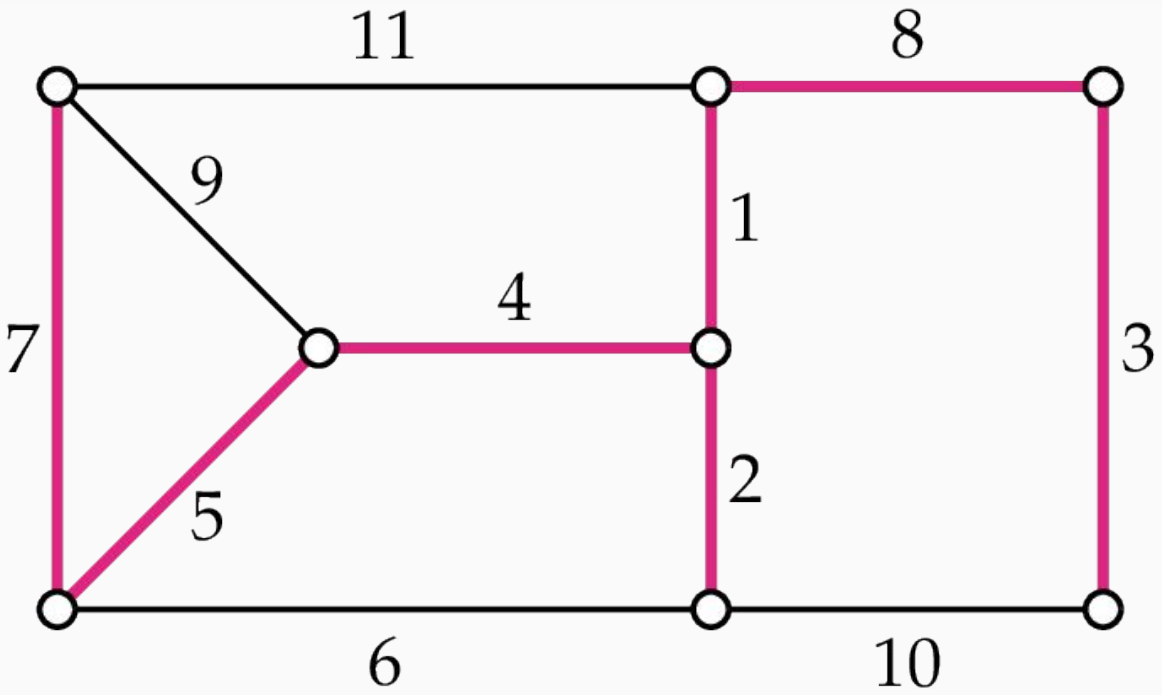
Kruskal’s algorithm



Kruskal’s algorithm



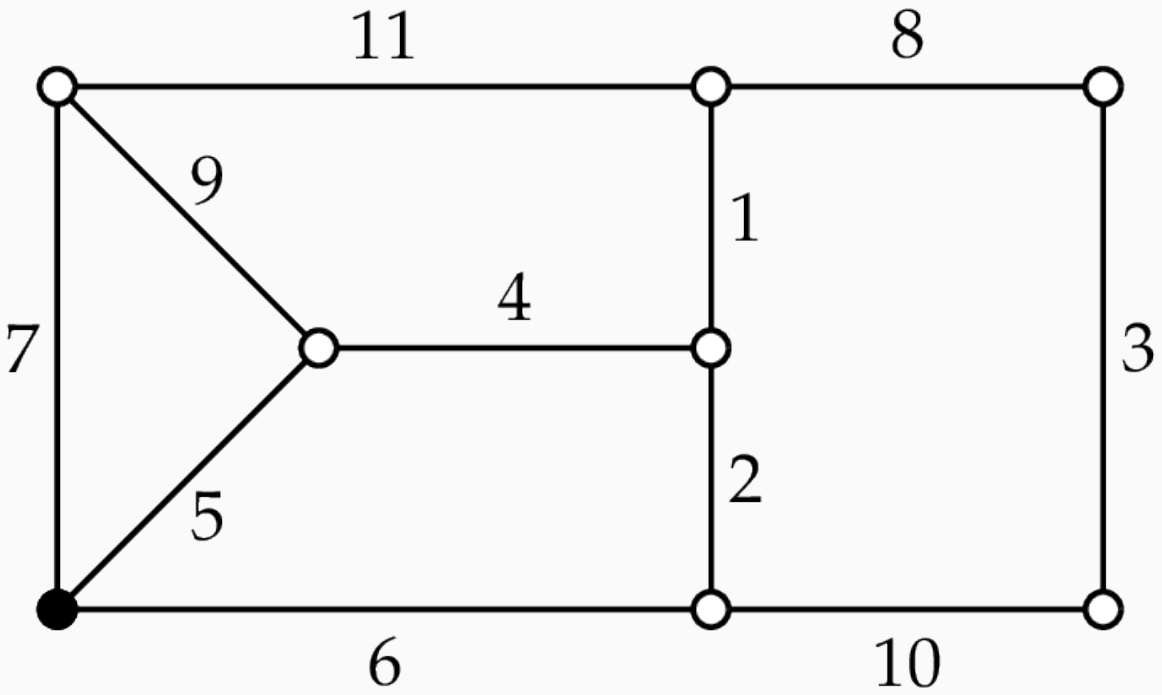
Kruskal’s algorithm



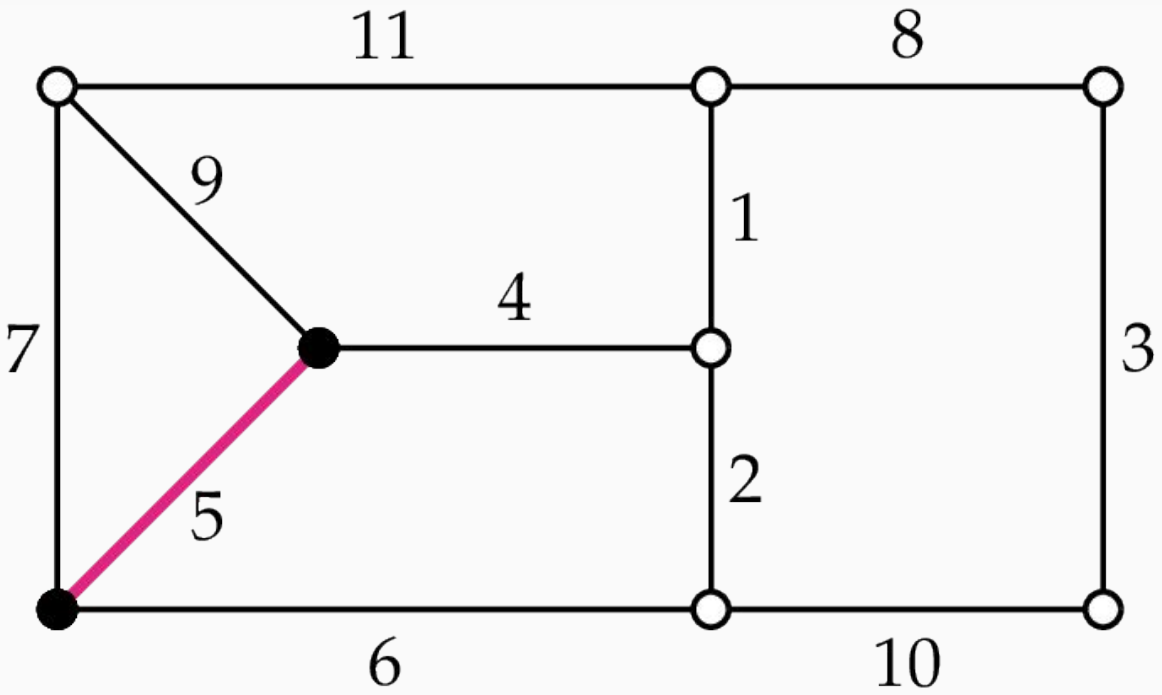
Prim’s algorithm

1. Pick any vertex **v**, and set **T** = {**v**}
2. Repeatedly “grow” **T** by adding smallest edge leaving **T**

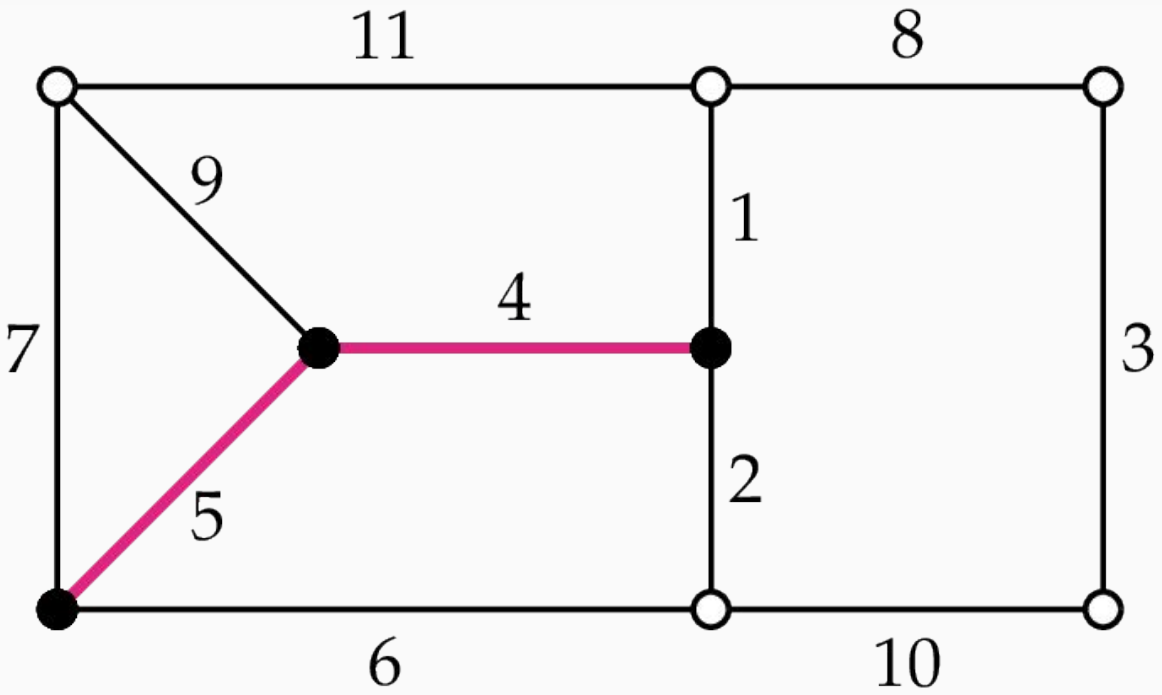
Prim’s algorithm



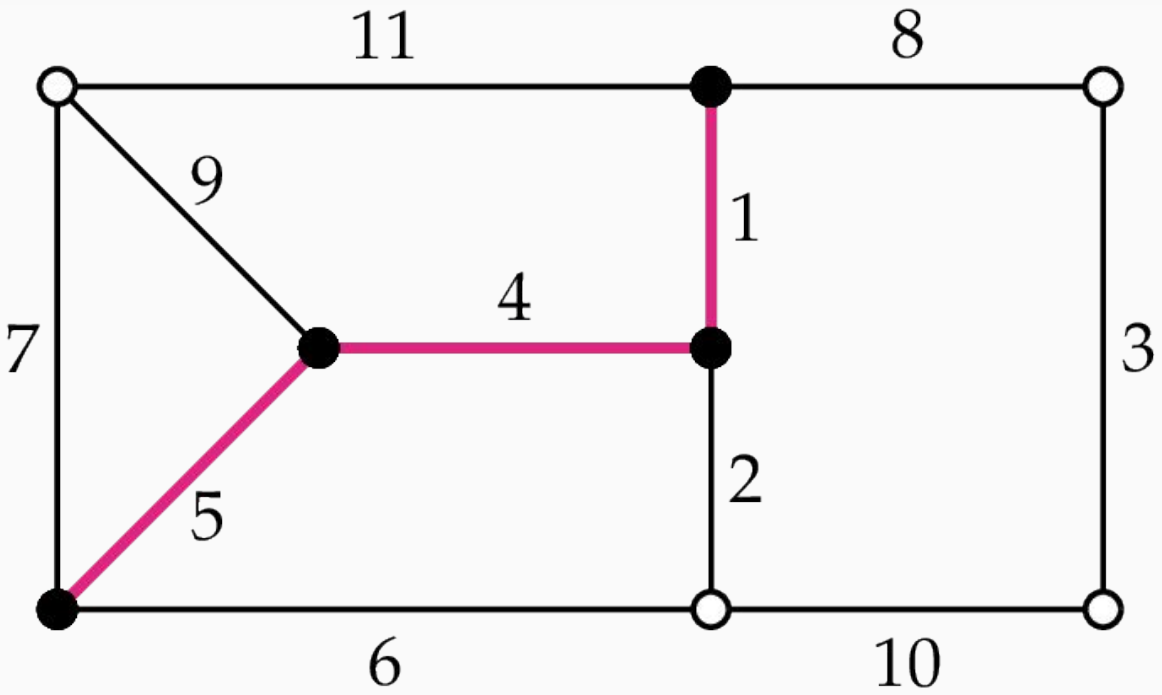
Prim’s algorithm



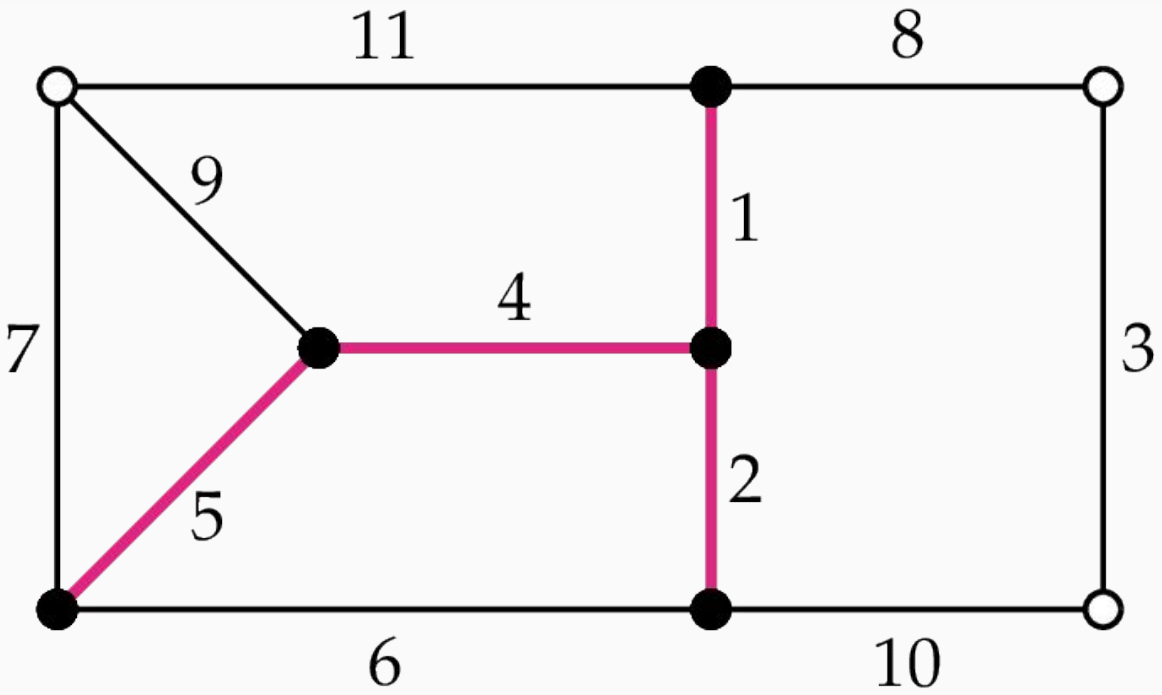
Prim’s algorithm



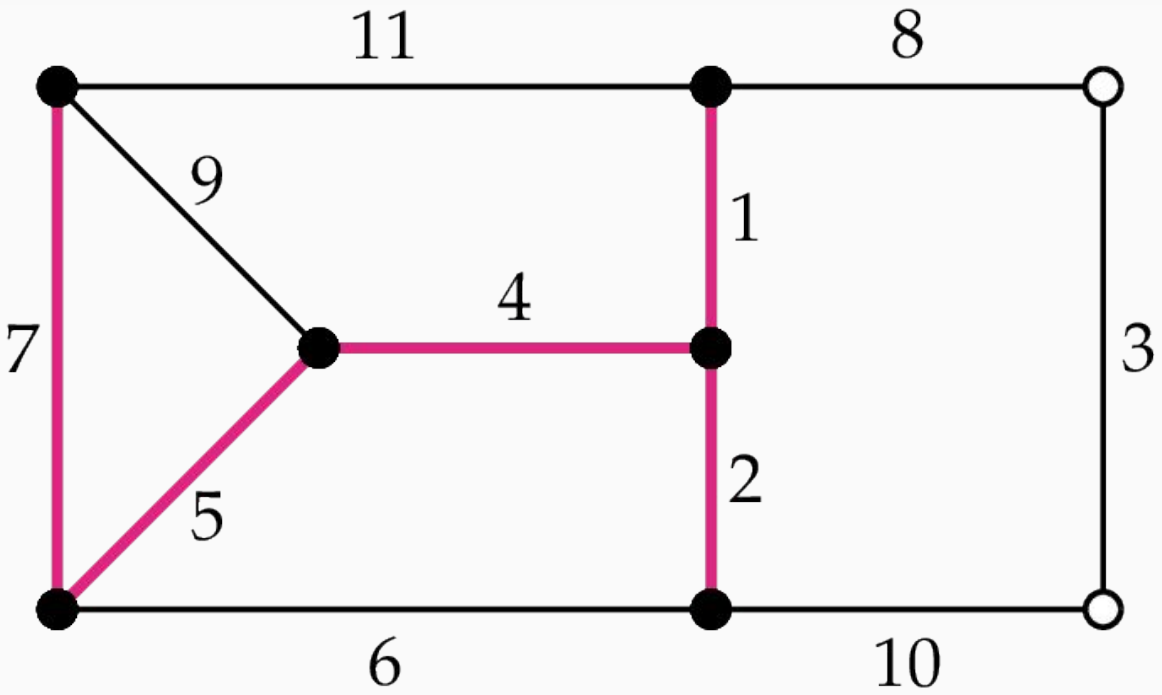
Prim’s algorithm



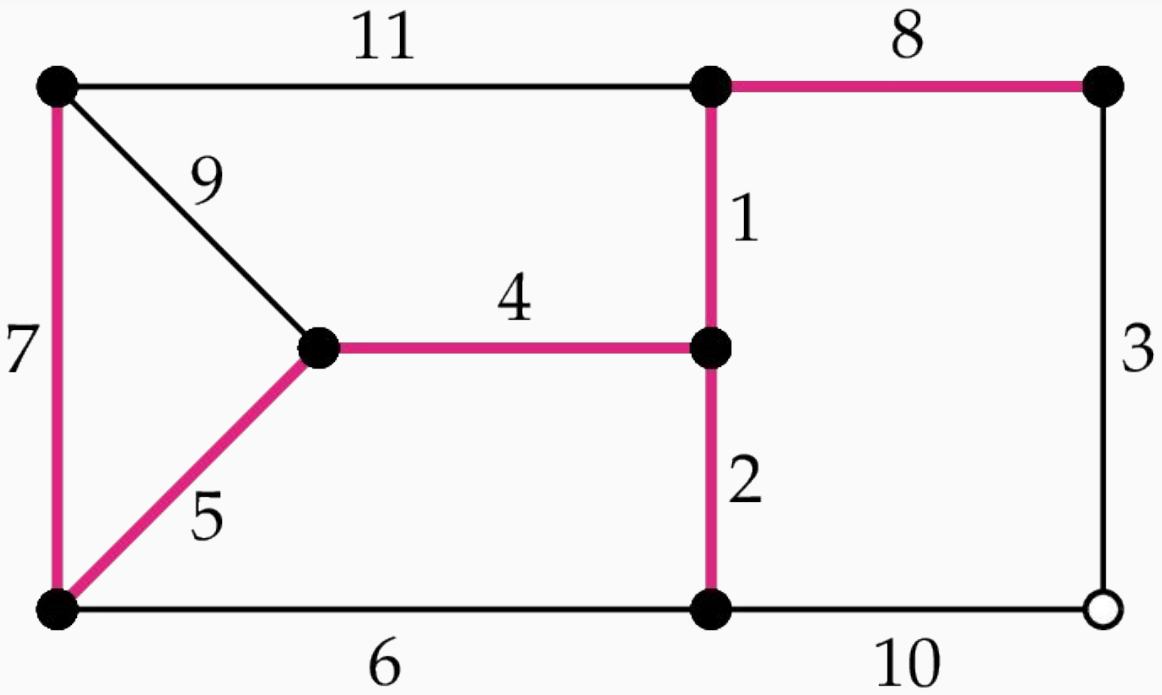
Prim’s algorithm



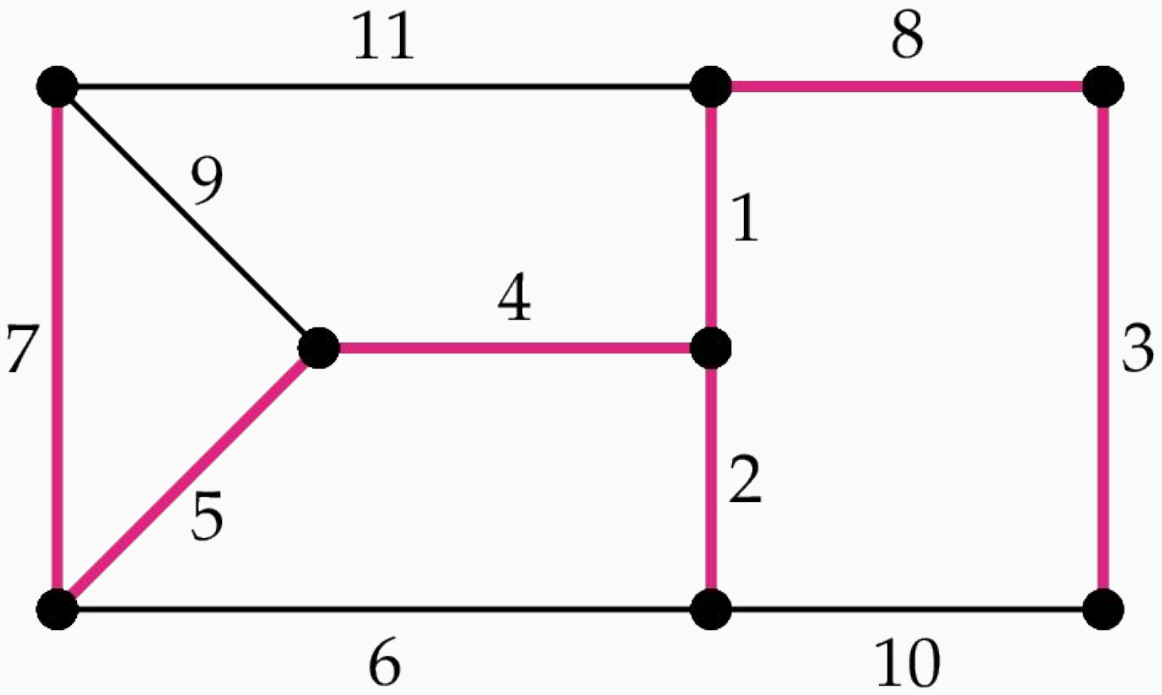
Prim’s algorithm



Prim’s algorithm



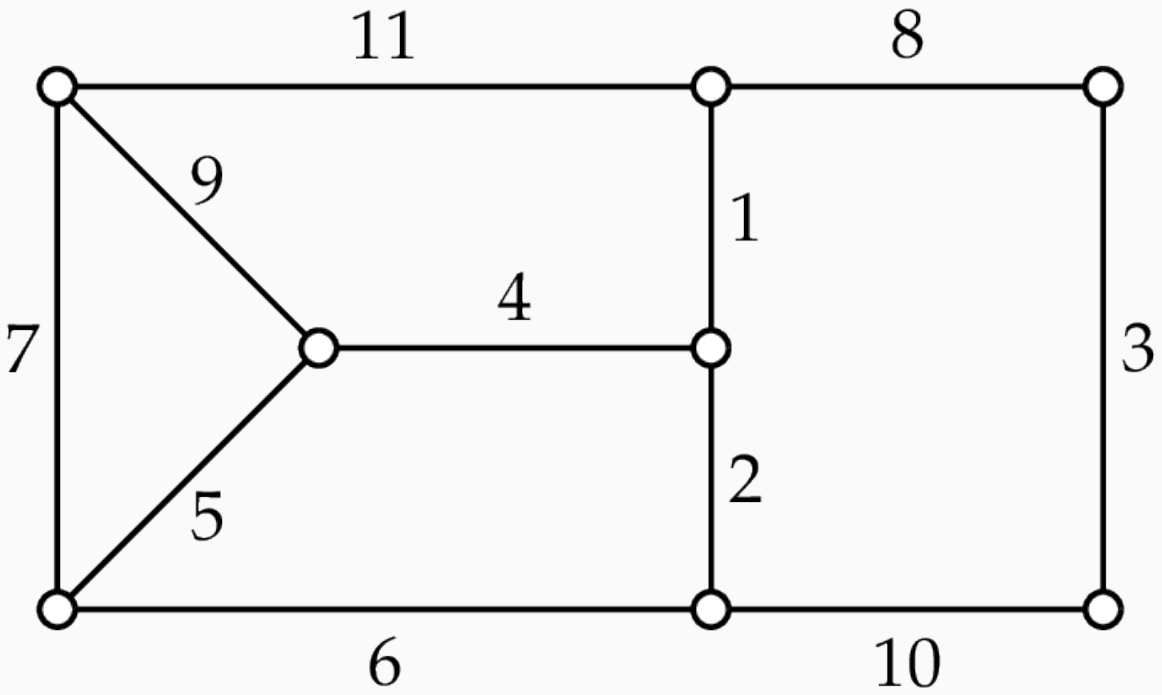
Prim’s algorithm



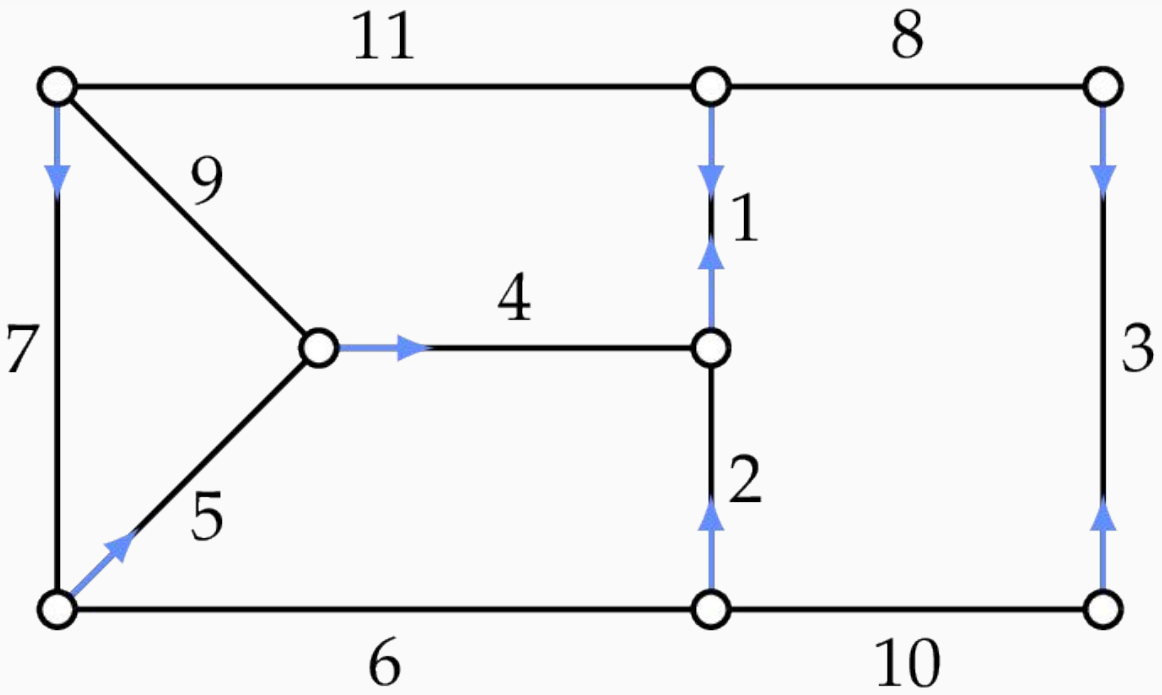
Boruvka’s algorithm

1. Repeat until one component:
    a. Identify each component’s safe edge.
    b. Add all safe edges.
    c. Relabel components of forest.

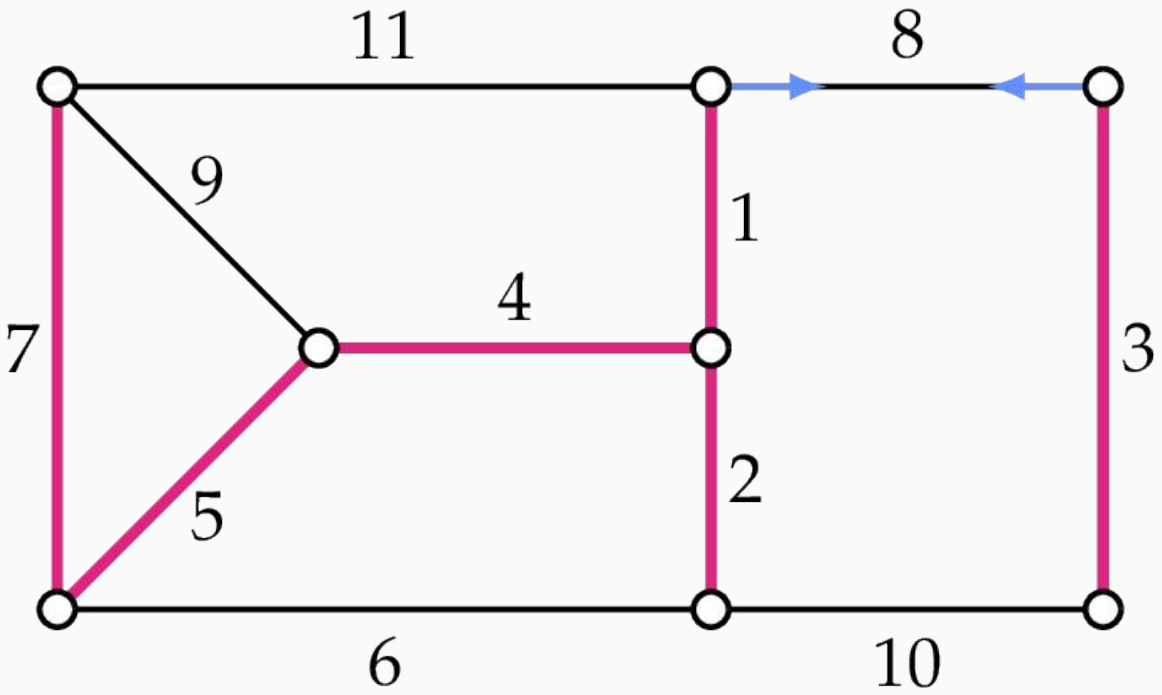
Boruvka’s algorithm



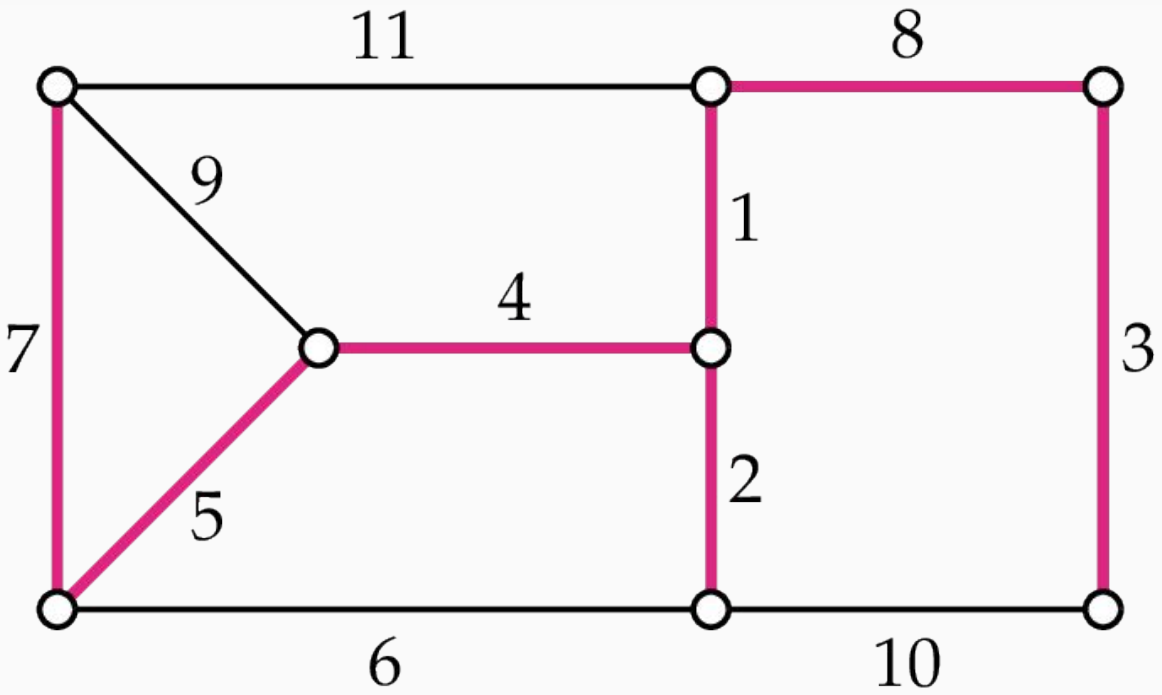
Boruvka's algorithm



Boruvka’s algorithm



Boruvka's algorithm



Boruvka’s algorithm

1. Repeat until one component:
    a. Identify each component’s safe edge.
    b. Add all safe edges.
    c. Relabel components of forest.

Boruvka’s algorithm

1. Repeat until one component:
    a. Identify each component’s safe edge.
    b. Add all safe edges.
    c. Relabel components of forest.

Boruvka’s algorithm

1. Repeat until one component:
   a. Identify each component’s safe edge.
   b. Add all safe edges.
   c. **Relabel components of forest.** (CountComp using DFS, etc.)

Boruvka’s algorithm

1. Repeat until one component:
   a. **Identify each component’s safe edge.**
   b. **Add all safe edges.**
   c. Relabel components of forest.

Boruvka’s algorithm

1. Repeat until one component:
   a. Identify each component’s safe edge.
   b. Add all safe edges.
   c. Relabel components of forest.

Store safe edges in array indexed by component number.

Boruvka's algorithm

1. Repeat until one component:
   a. **Identify each component's safe edge.**
   b. Add all safe edges.
   c. Relabel components of forest.

Linear scan through edges (***u***, ***v***):

- "Are ***u*** and ***v*** in different components?"

Boruvka's algorithm

1. Repeat until one component:
   a. **Identify each component's safe edge.**
   b. Add all safe edges.
   c. Relabel components of forest.

Linear scan through edges (***u***, ***v***):

- "Are ***u*** and ***v*** in different components?"
  - Check if they're the new minimum for both components.

Boruvka's algorithm - finding safe edges

$\textsc{AddSafeEdges}(E, F, count)$:

initialize $S[1, \ldots, count]$

for each $(u, v) \in E$

if $comp(u) \neq comp(v)$

if $w(u, v) < w(S[comp(u)])$

$S[comp(u)] = (u, v)$

if $w(u, v) < w(S[comp(v)])$

$S[comp(v)] = (u, v)$

for $i = 1, \ldots, count$

add $S[i]$ to F

Boruvka’s algorithm

1. Repeat until one component:
    a. Identify each component’s safe edge. (linear scan)
    b. Add all safe edges. (adding ***k*** edges)
    c. Relabel components of forest. (DFS or something)

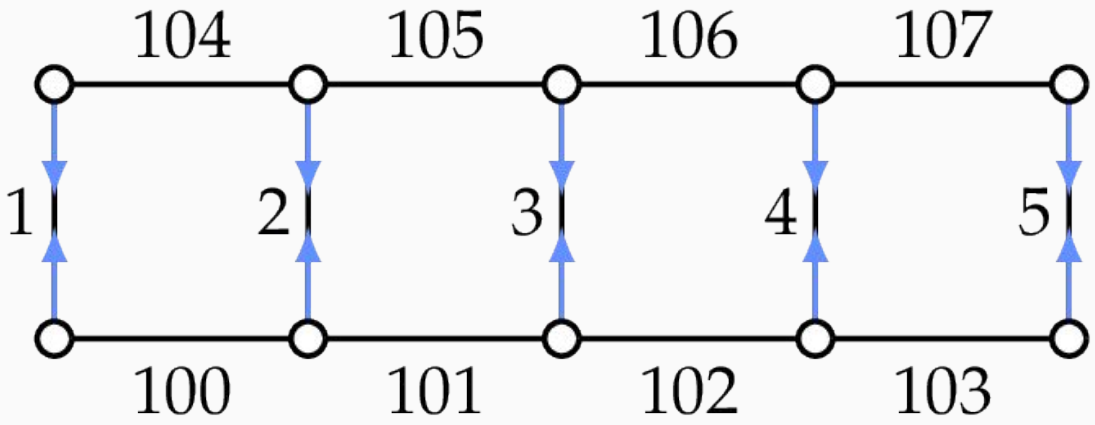
Boruvka's algorithm

1. Repeat until one component:
    a. Identify each component's safe edge. (linear scan) $O(|\boldsymbol{E}|)$
    b. Add all safe edges. (adding $\boldsymbol{k}$ edges) $O(|\boldsymbol{V}|)$
    c. Relabel components of forest. (DFS or something) $O(|\boldsymbol{E}|)$

Each iteration takes $O(|\boldsymbol{E}|)$ time. How many iterations?

Slowly shrinking components

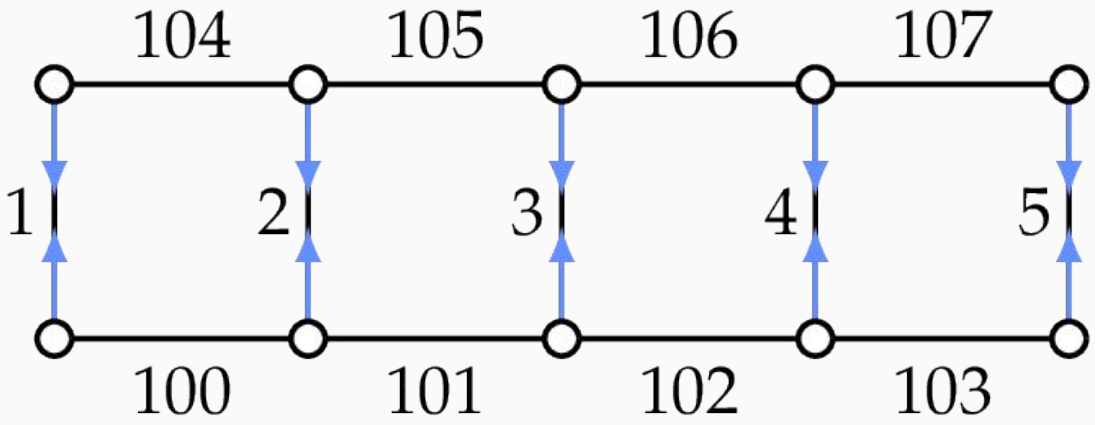
In the worst case, components are paired up, decreasing by a factor of 2.



Slowly shrinking components

In the worst case, components are paired up, decreasing by a factor of 2.

How long until there's only one component left?

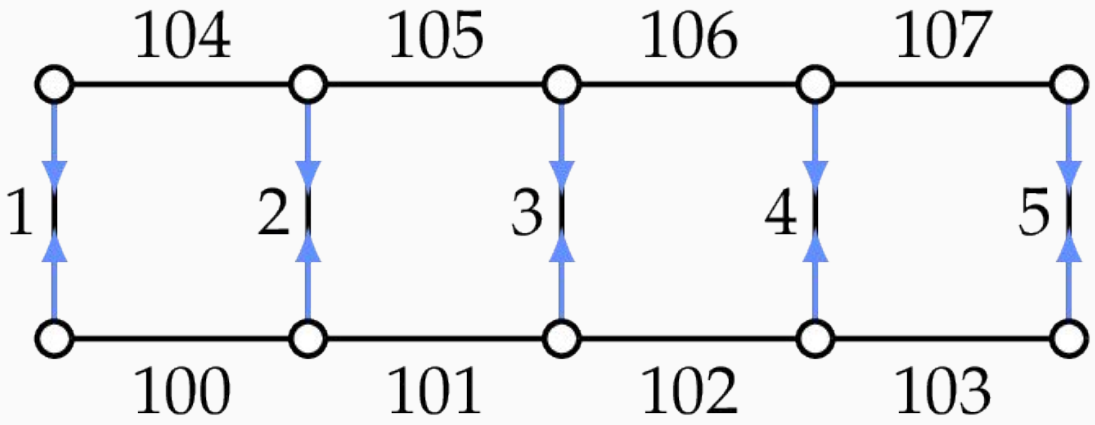


Slowly shrinking components

In the worst case, components are paired up, decreasing by a factor of 2.

How long until there's only one component left?

O(log |*V*|) iterations



Boruvka's algorithm

1. Repeat until one component:
    a. Identify each component's safe edge. (linear scan) **O(|*E*|)**
    b. Add all safe edges. (adding ***k*** edges) **O(|*V*|)**
    c. Relabel components of forest. (DFS or something) **O(|*E*|)**

Each iteration takes O(|***E***|) time. How many iterations? **O(log |*V*|)**

Total runtime: O(|***E***| log |***V***|)

Kruskal’s algorithm

1. Sort all the edge weights.
2. Repeatedly pick the smallest edge **without creating a cycle**.

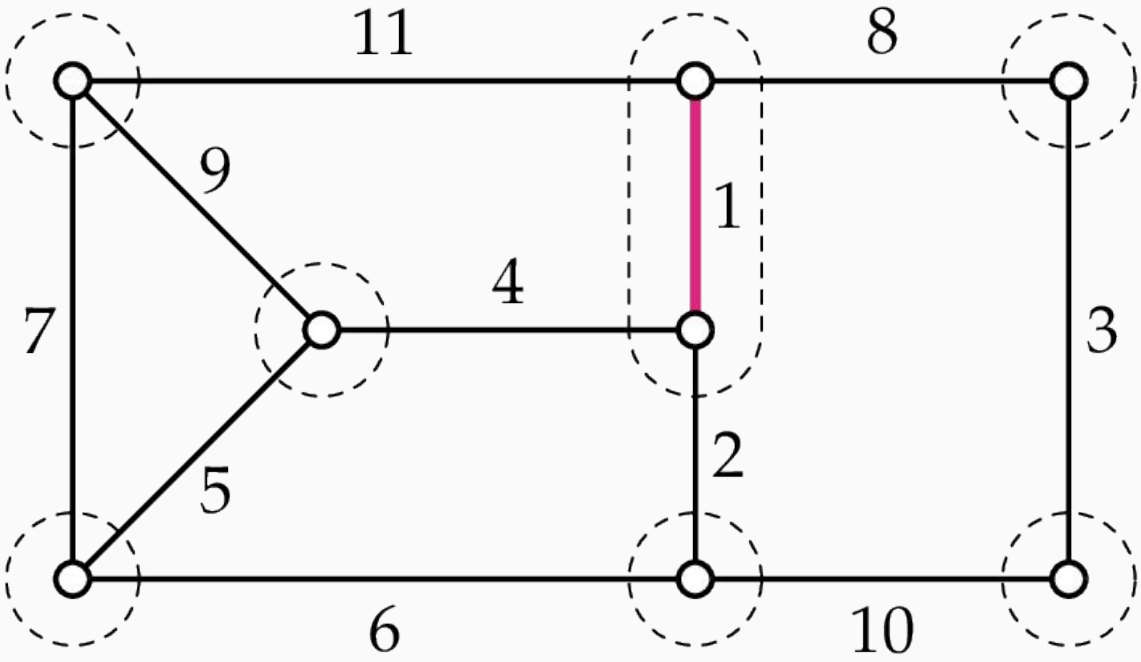
Kruskal’s algorithm

1. Sort all the edge weights.
2. Repeatedly pick the smallest edge **without creating a cycle**.

This step is about detecting components. Should we use CountComp again?

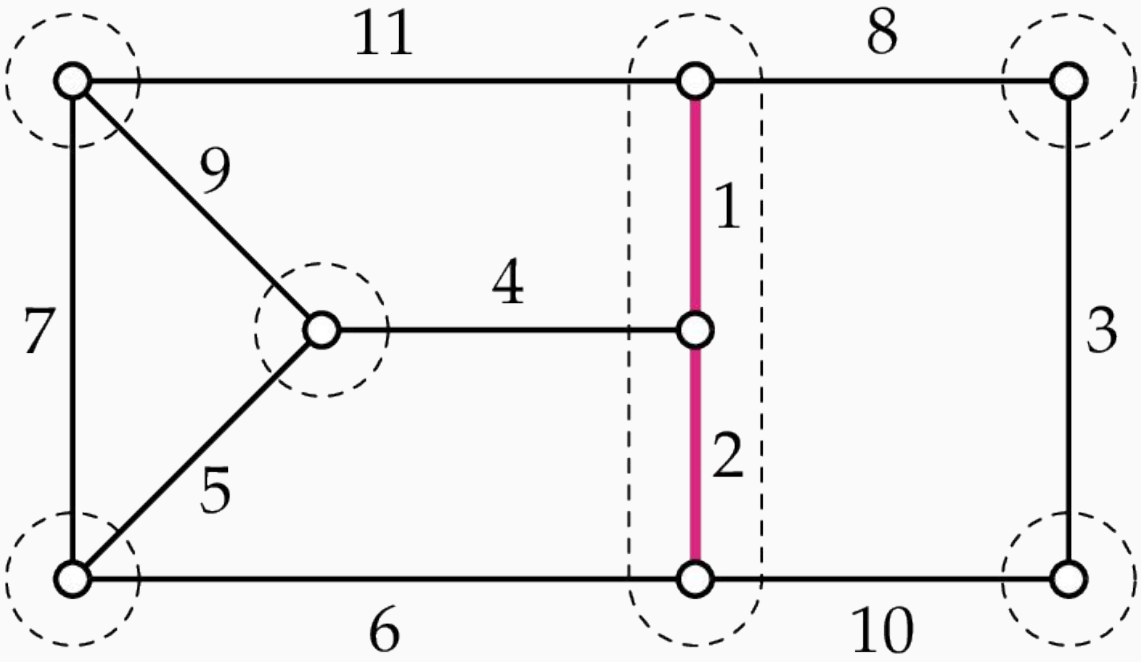
Kruskal’s algorithm

For each iteration of Kruskal, most components don’t change.



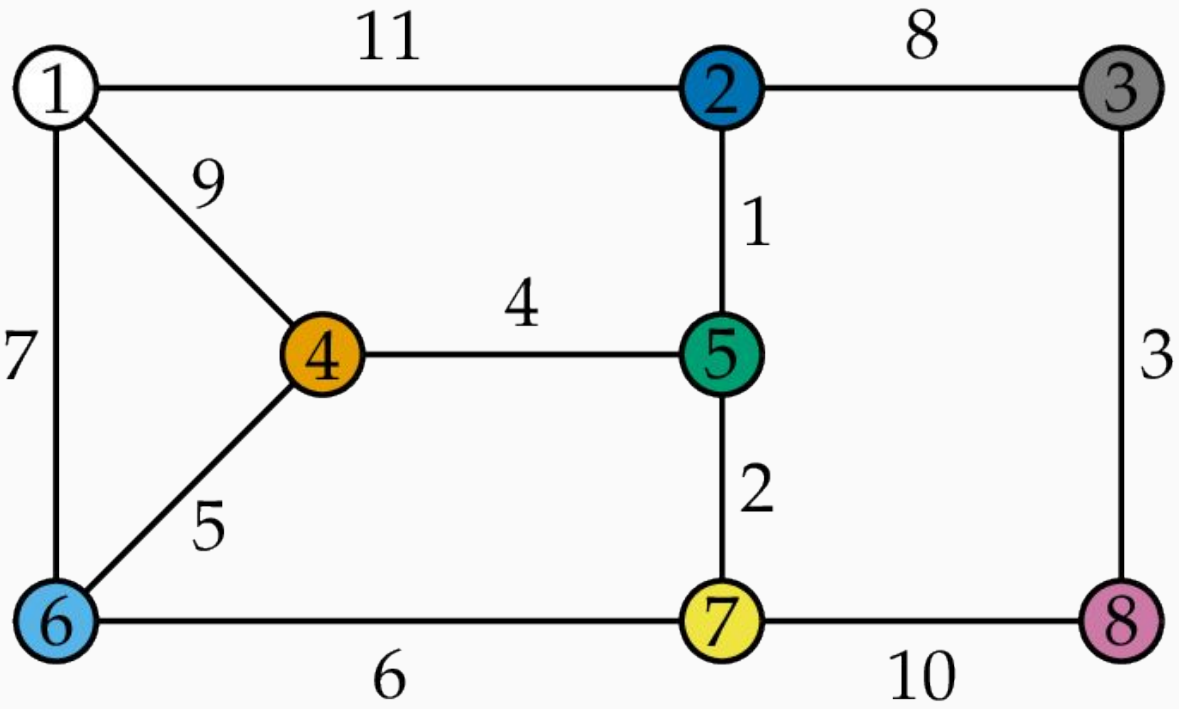
Kruskal’s algorithm

For each iteration of Kruskal, most components don’t change.



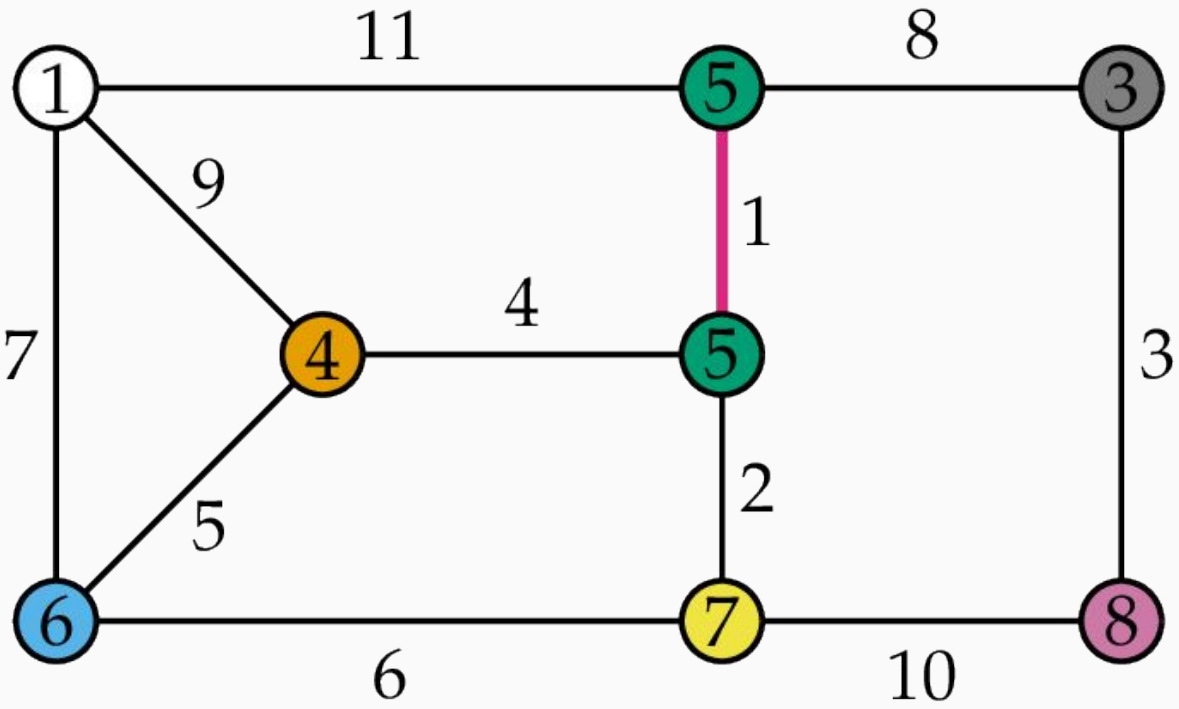
Kruskal's algorithm

Just relabel ***one of the merged components***.



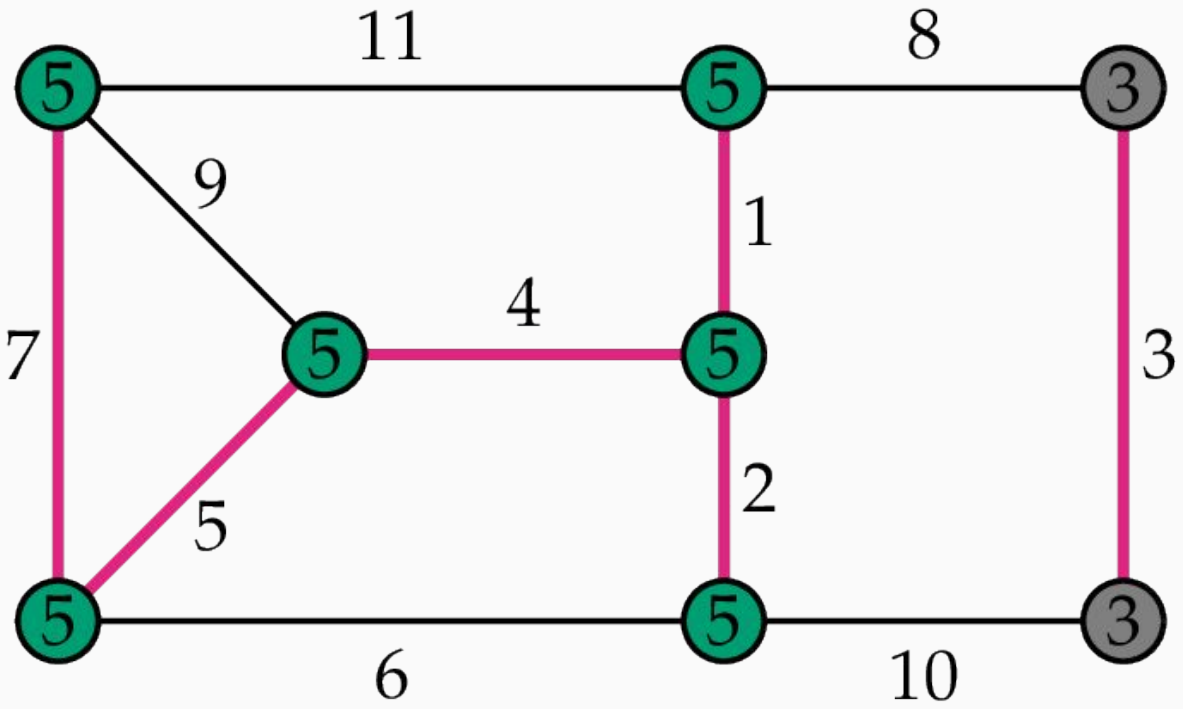
Kruskal's algorithm

Just relabel ***one of the merged components***.



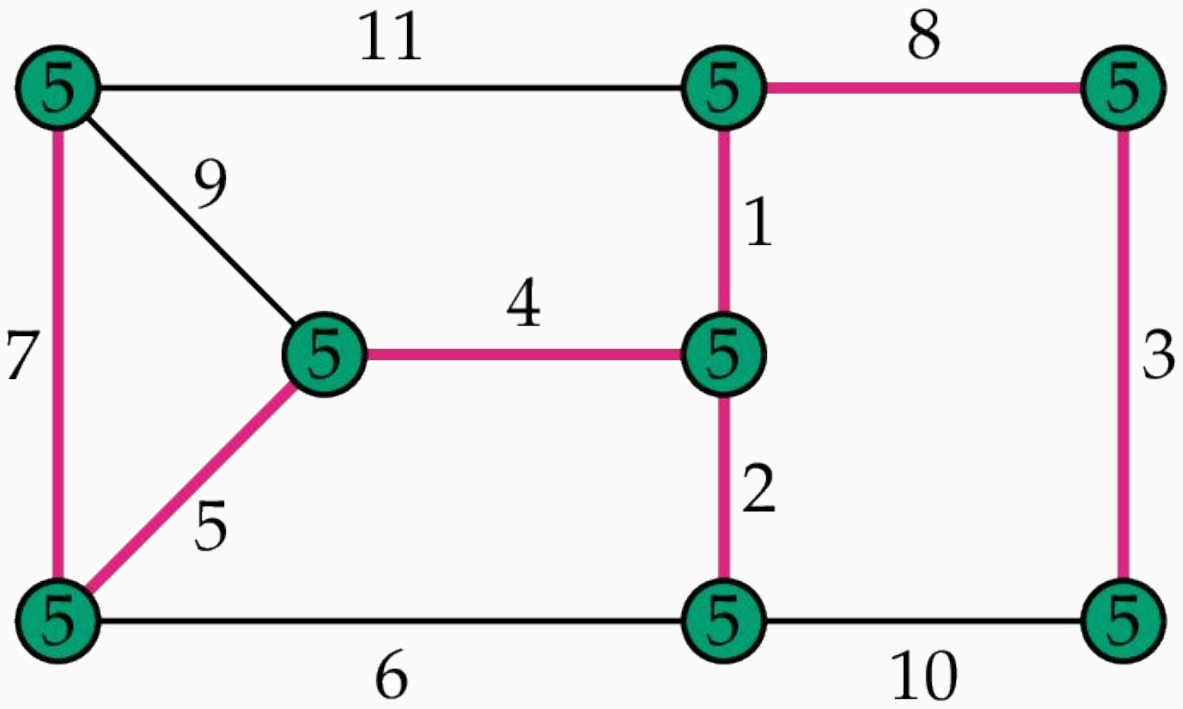
Kruskal's algorithm

Just relabel ***one of the merged components***.



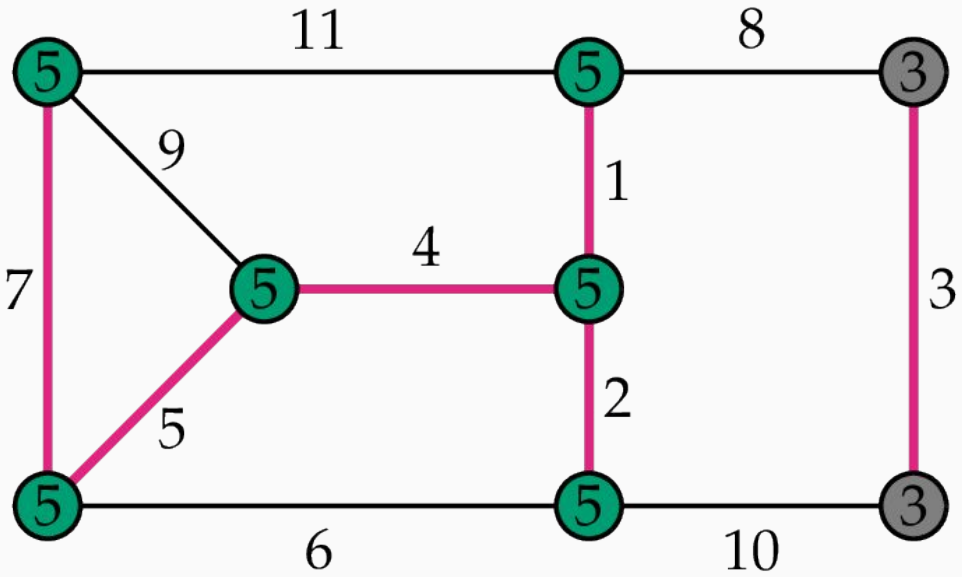
Kruskal's algorithm

Just relabel ***smaller of the two merged components***.



Kruskal’s algorithm

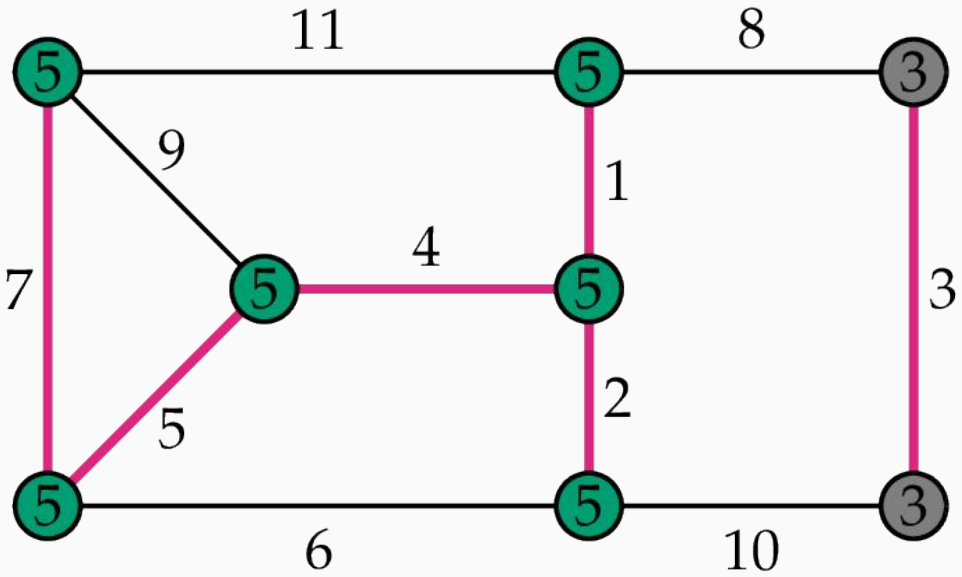
How long does relabeling take?



Kruskal's algorithm

How long does relabeling take?

- How many times can a vertex be relabeled?

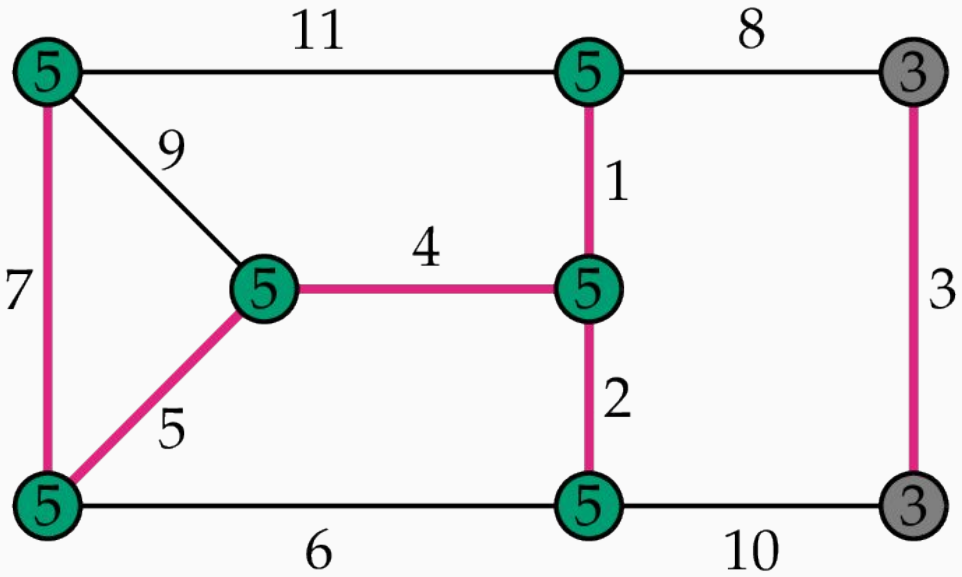


Kruskal's algorithm

How long does relabeling take?

- How many times can a vertex be relabeled? **O(log |*V*|) times**

O(|***V***| log |***V***|) in total



Kruskal’s algorithm

1. Sort all the edge weights.
2. Repeatedly pick the smallest edge **without creating a cycle**.

Kruskal’s algorithm

1. Sort all the edge weights. $O(|\boldsymbol{E}| \log |\boldsymbol{E}|)$
2. Repeatedly pick the smallest edge **without creating a cycle**. $O(|\boldsymbol{E}| + |\boldsymbol{V}| \log |\boldsymbol{V}|)$

Kruskal's algorithm

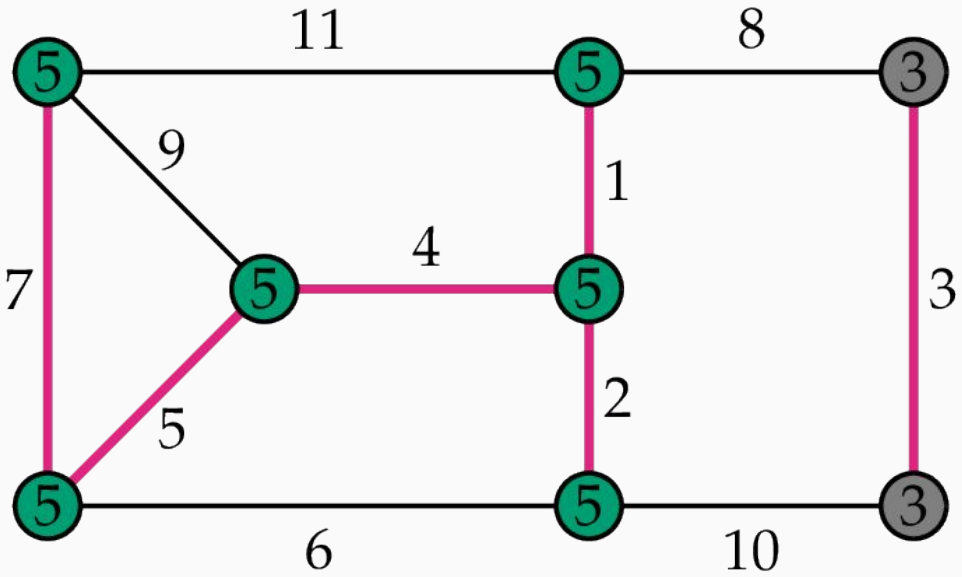
1. Sort all the edge weights. $O(|\boldsymbol{E}| \log |\boldsymbol{E}|)$
2. Repeatedly pick the smallest edge **without creating a cycle**. $O(|\boldsymbol{E}| + |\boldsymbol{V}| \log |\boldsymbol{V}|)$

Total: $O(|\boldsymbol{E}| \log |\boldsymbol{V}|)$

Kruskal's algorithm

Kruskal's algorithm often described using a **disjoint-set data structure**:

- MakeSet(***v***)
- Find(***v***)
- Union(***u***, ***v***)



Kruskal's algorithm

```
KRUSKAL(V, E):
    sort E by weight
    F := GRAPH(V, ∅)
    for each v ∈ V
        MAKESET(v)
    for each (u, v) ∈ E
        if FIND(u) ≠ FIND(v)
            UNION(u, v)
            add (u, v) to F
    return F
```

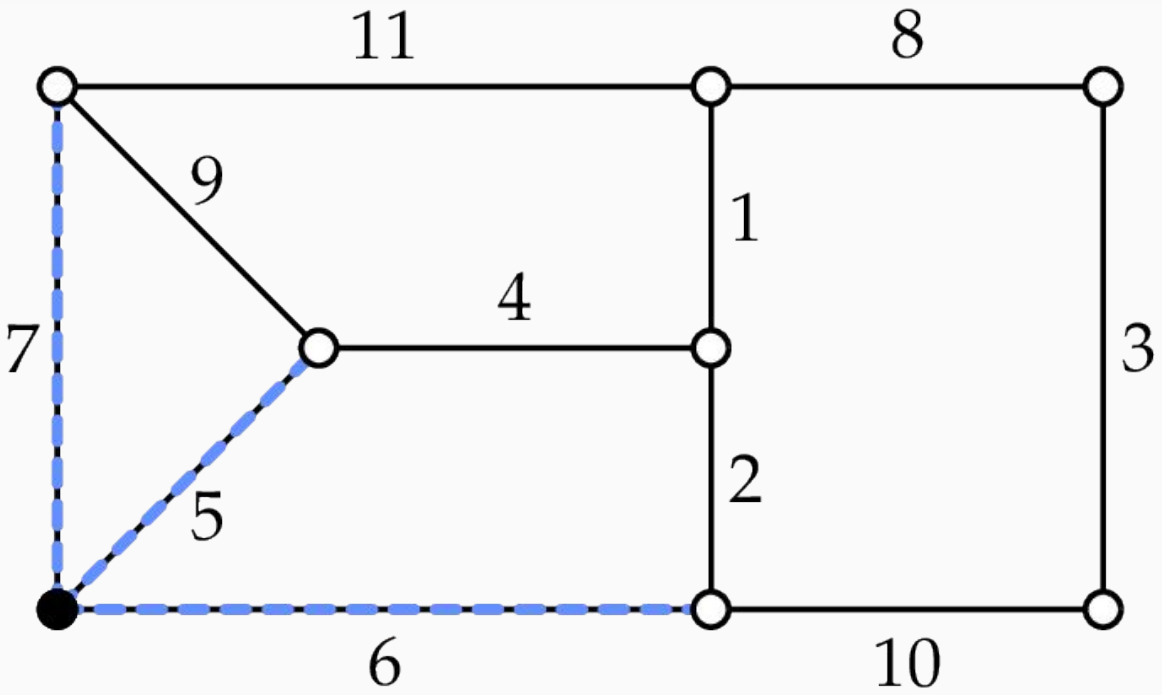
Prim’s algorithm

1. Pick any vertex **v**, and set **T** = {**v**}
2. Repeatedly “grow” **T** by adding smallest edge leaving **T**

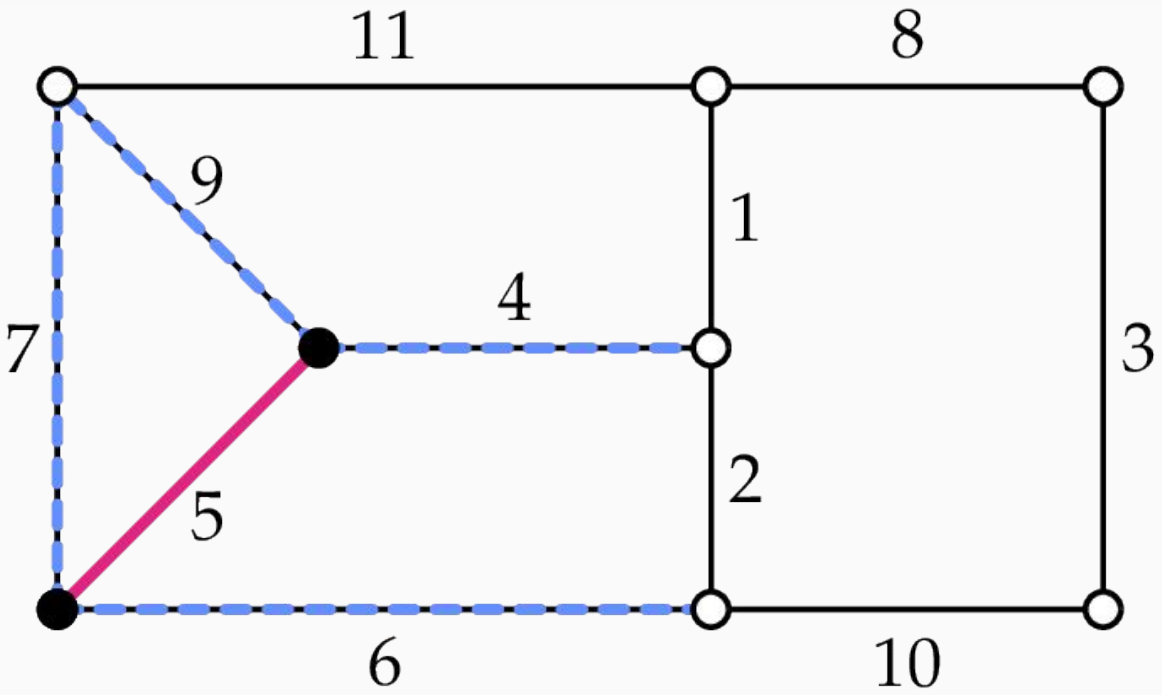
Prim’s algorithm

1. Pick any vertex **v**, and set **T** = {**v**}
2. Repeatedly “grow” **T** by adding smallest edge leaving **T**
    a. Something-first search where bag = priority queue storing edges

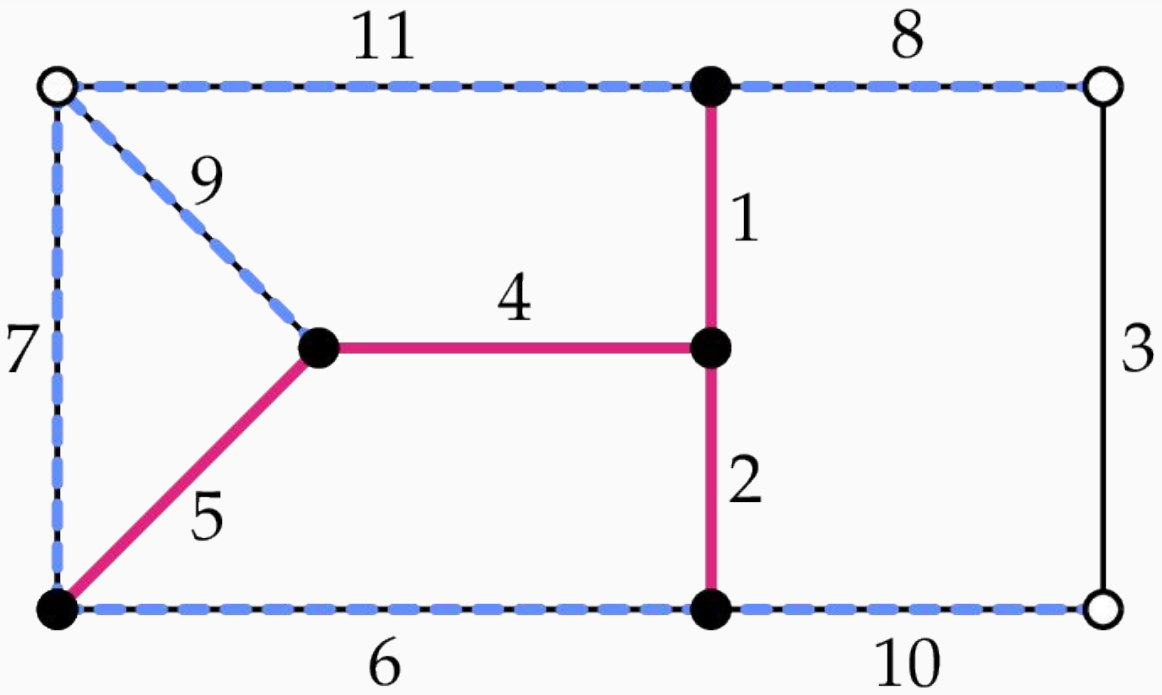
Prim’s priority queue



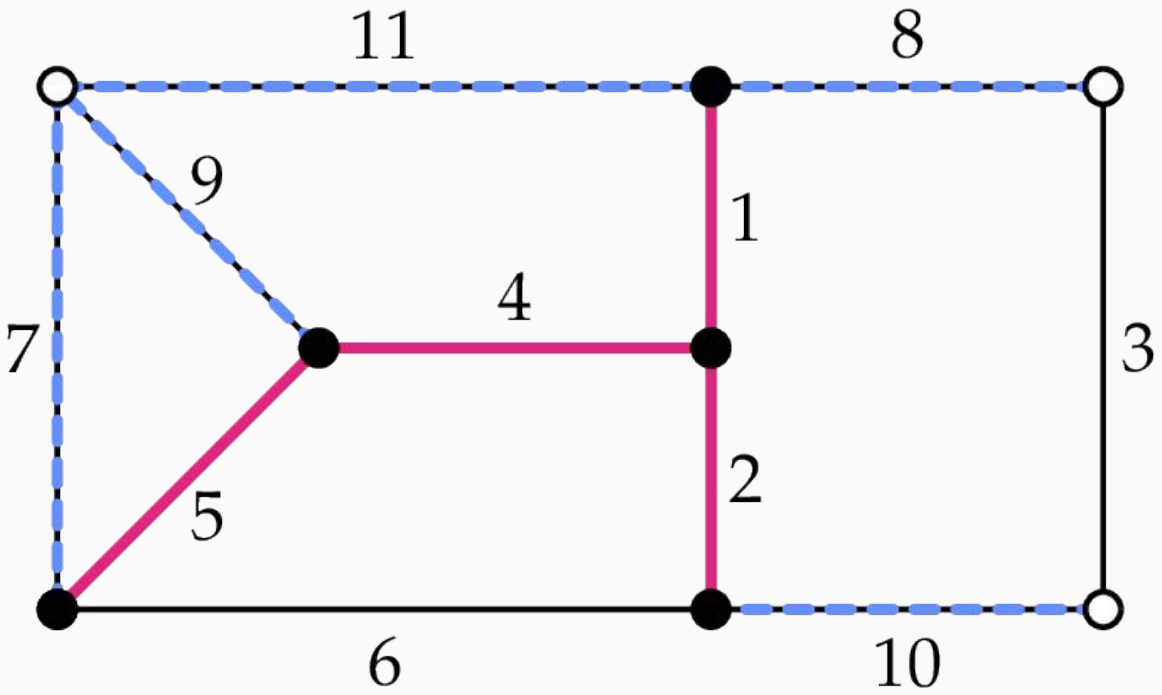
Prim’s priority queue



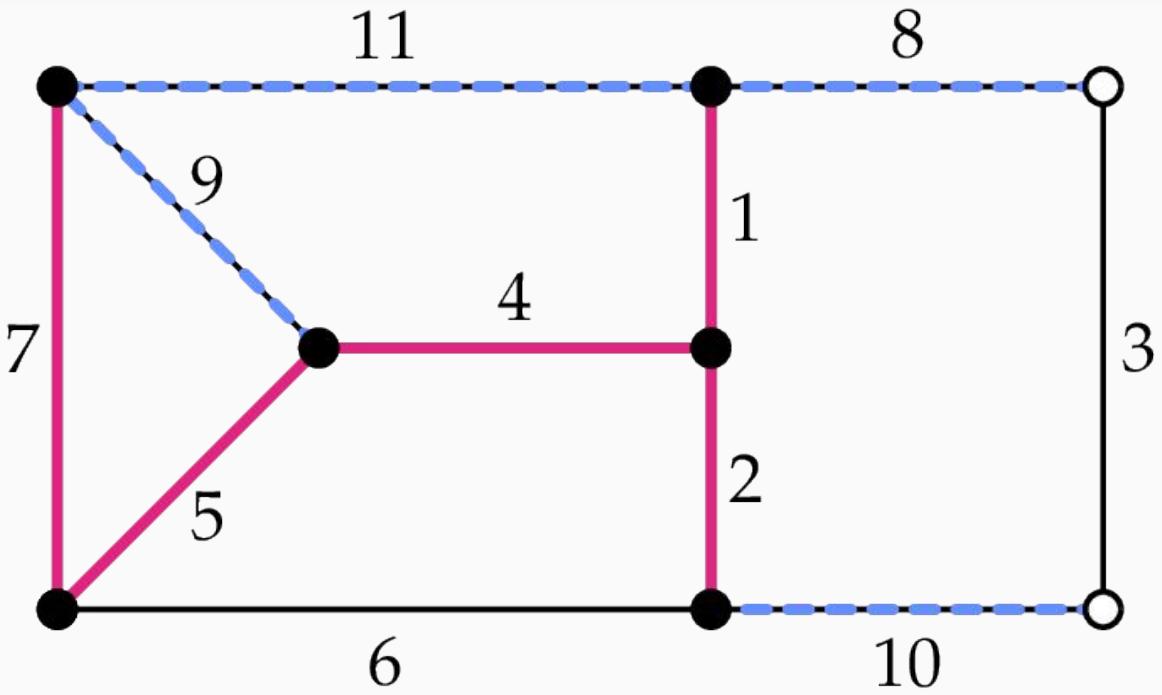
Prim’s priority queue



Prim’s priority queue



Prim’s priority queue



Prim's algorithm

```
PRIM(V, E):
    mark arbitrary vertex r
    H := HEAP()
    for each (r, v) ∈ E
        H.INSERT((r, v), w(r, v))
    F := GRAPH(V, ∅)
    while |H| > 0
        (u, v), w := H.EXTRACTMIN()
        if v is unmarked
            mark v
            add (u, v) to F
            for each (v, v′) ∈ E
                H.INSERT((v, v′), w(v, v′))
    return F
```

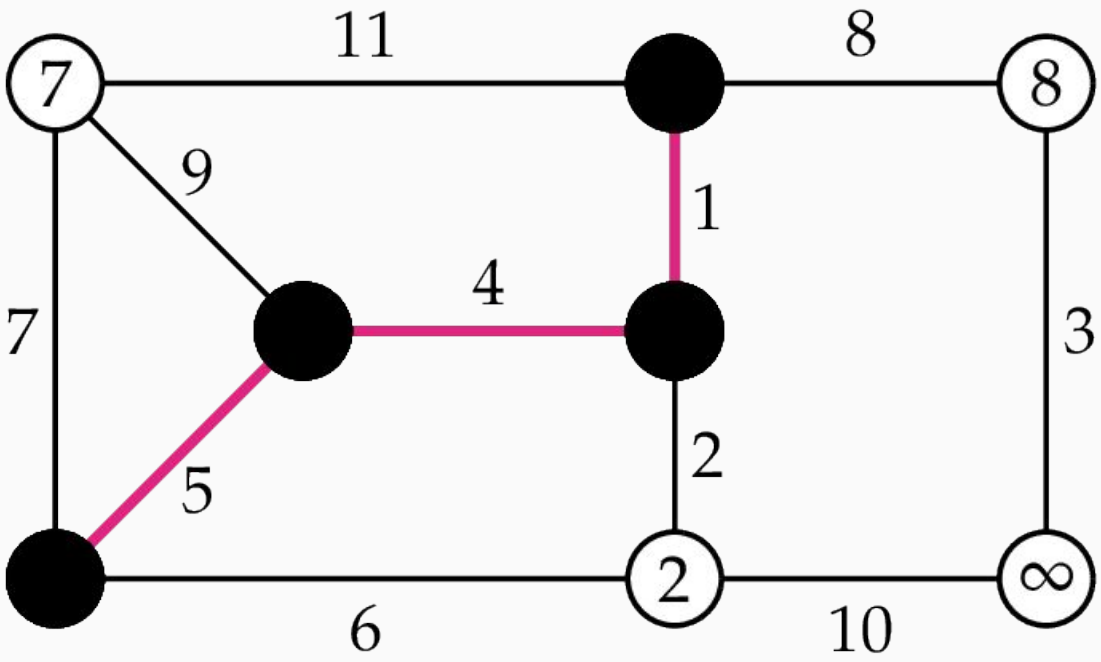
Prim's algorithm

1. Pick any vertex **v**, and set **T** = {**v**}
2. Repeatedly "grow" **T** by adding smallest edge leaving **T**
    a. Something-first search (bag = priority queue/heap)

O(|**E**| log |**V**|) as well

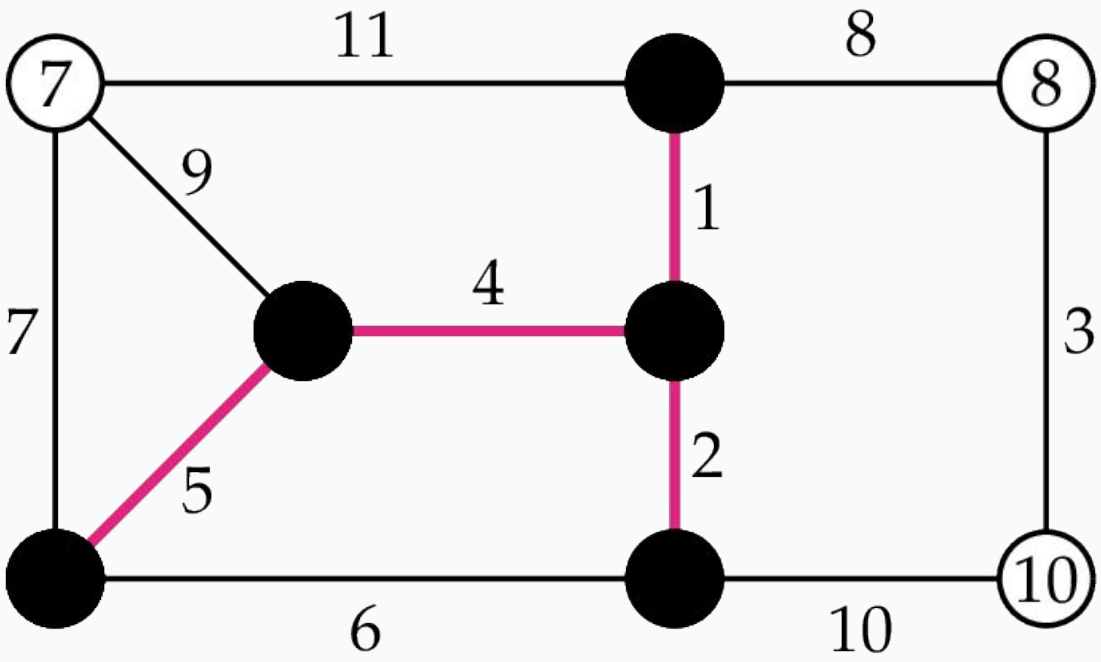
A vertex-centric version

An alternate implementation of Prim's algorithm uses ***vertex*** priorities

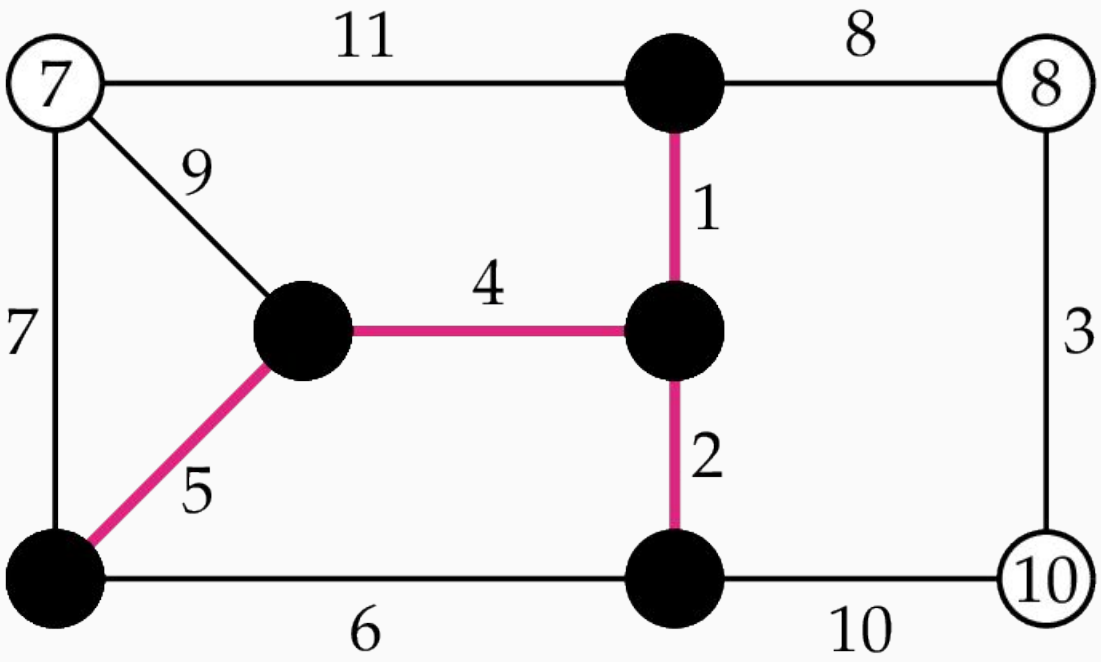


A vertex-centric version

An alternate implementation of Prim's algorithm uses ***vertex*** priorities



A vertex-centric version

An alternate implementation of Prim's algorithm uses ***vertex*** priorities

We'll see almost the same idea in Dijkstra's algorithm